nabia C++: Methods and Reference

The numerical library, its command-line application and its Python bindings

nabia documentation

October 2026

Abstract

This document is the reference for the C++ project of nabia (`cpp/`). It describes every method the library implements, stated mathematically, with proofs where a property is relied on:

- time series containers, signal filters, forecasters (exponential smoothing, Kalman filter, Gaussian processes) and five optimizers;
- risk measures, risk budgeting, the walk-forward portfolio engine and its metrics;
- the elastic net factor regression engine;
- the command-line application, and the pybind11 bindings that build the `nabia_python_bridge` wheel for every Python version and platform.

The Python package built on the wheel and the web application with its agents have their own references (`python/doc` and `webui/doc`). Each concept points to the code that implements it, by file and line range, and the key code is shown in excerpts *extracted from the source when the document is built*, so a renamed function stops the build instead of leaving a stale reference.

Contents

I Overview **3**

1 Purpose **3**

2 Architecture **3**
2.1 Three projects . . . 3
2.2 Layers of the C++ project . . . 4
2.3 Directory map . . . 4
2.4 Build . . . 4

II The numerical core (C++) **5**

3 Time series containers **5**

4 Signal extraction **5**
4.1 Exponentially weighted moving average . . . 6
4.2 Butterworth band-pass . . . 7

5 Forecasting **7**
5.1 Simple exponential smoothing . . . 7
5.2 Kalman filter . . . 8
5.3 Gaussian process regression . . . 9

6 Optimization **10**
6.1 L-BFGS . . . 10
6.2 Augmented Lagrangian . . . 12
6.3 Sequential quadratic programming . . . 13
6.4 Nelder–Mead with box bounds . . . 15
6.5 Differential evolution . . . 16

7 Risk measures **18**

8 Risk budgeting **19**

9 The walk-forward portfolio engine **20**
9.1 Timing and absence of look-ahead . . . 20
9.2 Covariance estimation . . . 21
9.3 Allocation, budget schedule and volatility targeting . . . 22
9.4 Phase 1: parallel solves . . . 23
9.5 Phase 2: drift, trading and accounting . . . 24

10 Performance metrics **25**

11 Elastic net factor regression **26**
11.1 The elastic net . . . 26
11.2 The regression engine . . . 28

III Application and Python bindings **31**

12 Command-line application **31**

13 Backtest bindings for the Python package **31**

14 The full library in Python: `nabia_python_bridge` **32**
14.1 Modules . . . 32
14.2 Design . . . 33
14.3 pandas interface . . . 34
14.4 Development build and tests . . . 35
14.5 Wheels for each Python version and platform . . . 35

IV Verification and limitations **37**

15 Verification **37**

16 Limitations and future work **38**

Part I
Overview

1 Purpose

nabia is a platform for researching and running *systematic trend strategies* across equity indices, government bond futures, commodities and crypto. The C++ library is its numerical core: it extracts trends from noisy prices (§4, §5), turns risk budgets into positions with a deterministic, tested engine (§8, §9), measures the result (§10) and explains a target series with factors (§11). Everything is available to Python through the `nabia_python_bridge` wheel (§14).

2 Architecture

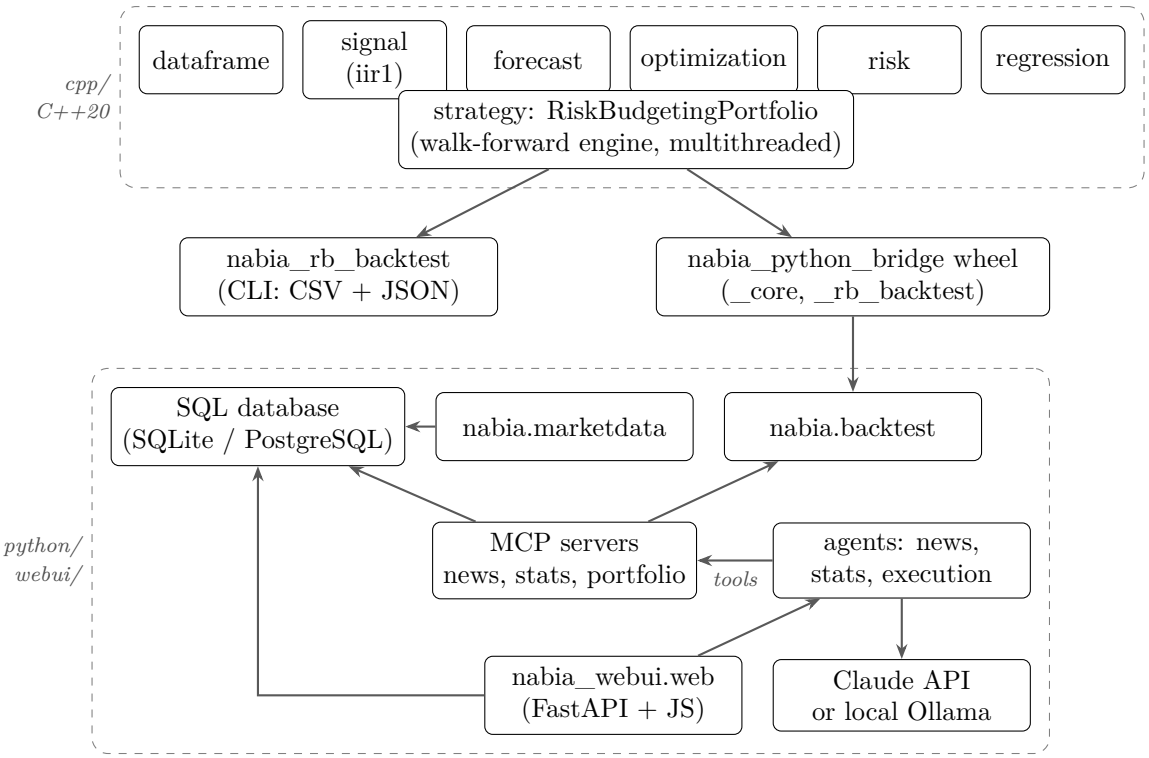


Figure 1: The whole system and its three projects. Arrows point from a component to what it uses.

2.1 Three projects

The repository holds three independent projects. Each opens on its own in an IDE, and each has its own documentation in a `doc` folder:

Folder	IDE	Contents
`cpp/`	CLion (CMake)	the C++ library (`include`, `src`), its tests, the command-line application, and the pybind11 bindings that build the `nabia_python_bridge` wheel into `dist/`
`python/`	PyCharm	the `nabia` package (backtests, analytics, market data) on top of the wheel, the examples, and the Python tests including the bridge tests
`webui/`	PyCharm	`nabia_webui`: the agents, their MCP servers and the web app, on top of the `nabia` package

The projects depend on each other only through installed packages: `cpp` builds the wheel, `python` installs it (`python/scripts/setup_env.sh`), and `webui` installs both the wheel and `python` in editable mode (`webui/scripts/setup_env.sh`). Shared data, the market database `data/market.db`, is at the repository root.

2.2 Layers of the C++ project

Library (`include/nabia`, `src`): the static library `nabia::nabia`, built with CMake. It uses C++20, Eigen for linear algebra and iir1 for filter design. Each module exposes an abstract interface whose parameter struct is owned through `std::unique_ptr`:

- `IFilter<P>` (`include/nabia/signal/IFilter.h:20-43`);
- `IForecaster<P>` (`include/nabia/forecast/IForecaster.h:38-81`);
- `IOptimizer<P,R>` (`include/nabia/optimization/IOptimizer.h:16-39`);
- `IRiskMeasure` (`include/nabia/risk/IRiskMeasure.h:18-43`).

Application `nabia_rb_backtest` (`apps/rb_backtest`): a backtest from a prices CSV and a JSON configuration, written to CSV (§12).

Python bindings (`bindings`): two pybind11 modules in one package, `nabia_python_bridge`. `_core` exposes the whole library (§14); `_rb_backtest` exposes the JSON-configured backtest of the application (§13). `pyproject.toml` builds them into a wheel (§14.5).

2.3 Directory map

Path (in `cpp/`)	Contents
`include/nabia/dataframe`	`TimeSeries`, `DataFrame`, the `TimeIndex` concept
`include/nabia/signal`	`IFilter`, `EWMA`, `ButterworthBandPass`, smoothing helpers
`include/nabia/forecast`	`IForecaster`, SES, Kalman filter, Gaussian process, kernels
`include/nabia/optimization`	`IOptimizer`; `convex/`: constraints, L-BFGS, augmented Lagrangian, SQP; `nongradopt/`: Nelder–Mead, differential evolution
`include/nabia/risk`	`IRiskMeasure`, volatility, VaR, ES, `RiskBudgeting<Measure>`
`include/nabia/stats`	descriptive statistics, normal density, CDF and quantile
`include/nabia/strategy`	`RiskBudgetingPortfolio`, performance metrics
`include/nabia/regression`	`ElasticNet`, `RegressionEngine`
`src`	the implementations, in the same layout
`tests`	Catch2 unit tests
`apps/rb_backtest`	command-line backtest (CSV and JSON)
`examples`	C++ usage examples
`bindings`	pybind11 sources (`src`) and the Python files of the wheel (`nabia_python_bridge`)
`pyproject.toml`, `scripts`	wheel build (scikit-build-core, cibuildwheel), `build_wheels.sh`
`doc`	this reference and the theory notes, LaTeX sources and PDFs

2.4 Build

Open `cpp/` in CLion, or use the CMake presets from that folder:

```
cmake --preset debug && cmake --build --preset debug    # library, tests, app, examples, docs
ctest --preset debug                                    # the Catch2 suite and the app example
cmake --build --preset debug --target nabia_docs        # only the PDFs of doc/
```

The presets are `debug`, `release`, `releaseWithDebugInfo`, `asan` and `python-bridge`; all treat warnings as errors. Dependencies are found on the system or fetched: Eigen, iir1, Catch2, nlohmann/json and pybind11. The `nabia_docs` target builds this reference and the theory notes

on exponential smoothing, the Kalman filter, Gaussian processes, convex optimization and elastic net regression (`doc/<area>/*.pdf`). For this document it first runs `doc/extract_snippets.py`, which writes the excerpts and the code locations.

Part II
The numerical core (C++)

3 Time series containers

All numerical code works on Eigen matrices, but financial data is indexed by time. `TimeSeries<Index>` and `DataFrame<Index>` pair an Eigen vector or matrix (rows are times) with a *strictly increasing* index:

- The index type satisfies the `TimeIndex` concept (copyable and totally ordered): calendar dates, timestamps or integers.
- Construction validates the shapes, the ordering and unique column names.
- Time lookups use binary search.

Alignment. Markets have different holidays. A cross-asset backtest keeps only the dates where every asset has a finite price.

Returns. Simple returns $r_t = P_t/P_{t-1} - 1$ (`pct_change`) or log returns $\log(P_t/P_{t-1})$ are computed between consecutive *kept* rows. A return can therefore span a removed holiday. That is the right convention for a portfolio that can only trade when all its markets are open.

Code 1: Alignment: keep rows where every column is finite
`include/nabia/dataframe/DataFrame.h:108-122`

```
[[nodiscard]] DataFrame drop_missing() const {
    std::vector<Index> index;
    std::vector<Eigen::Index> keep;
    for (Eigen::Index i = 0; i < rows(); ++i) {
        if (values_.row(i).allFinite()) {
            keep.push_back(i);
            index.push_back(index_[static_cast<std::size_t>(i)]);
        }
    }
    Eigen::MatrixXd out(static_cast<Eigen::Index>(keep.size()), cols());
    for (std::size_t k = 0; k < keep.size(); ++k) {
        out.row(static_cast<Eigen::Index>(k)) = values_.row(keep[k]);
    }
    return DataFrame(std::move(index), columns_, std::move(out));
}
```

Code

`TimeIndex` concept — `include/nabia/dataframe/TimeIndex.h:17`
`TimeSeries` — `include/nabia/dataframe/TimeSeries.h:47-106`
`DataFrame` — `include/nabia/dataframe/DataFrame.h:23-146`
simple returns — `include/nabia/dataframe/DataFrame.h:126-132`
log returns — `include/nabia/dataframe/DataFrame.h:134-140`

4 Signal extraction

Filters implement `IFilter<P>`. They are *causal*: output t uses inputs up to t only. Two modes are supported:

- streaming: `process(x)`, one sample at a time, keeping state;
- batch: `apply(signal)`, from a fresh state, without disturbing the streaming state.

Filter design is delegated to iir1, behind a private implementation struct (pimpl), so iir1 never appears in public headers.

4.1 Exponentially weighted moving average

The EWMA is the one-pole low-pass

$$y_t = \alpha x_t + (1-\alpha) y_{t-1}, \qquad H(z) = \frac{\alpha}{1-(1-\alpha)z^{-1}}, \qquad \alpha \in (0, 1].$$

Its DC gain is $H(1) = 1$, so a constant input passes unchanged.

Proposition 4.1 (Seeding)**.** *Let $\tilde{y}$ be the EWMA of $x_t - x_0$ started from a zero state. Then $y_t = x_0 + \tilde{y}_t$ equals the recursion started at $y_{-1} = x_0$:*

$$y_t = \alpha \sum_{k=0}^{t} (1-\alpha)^k x_{t-k} + (1-\alpha)^{t+1} x_0.$$

Proof. Unrolling the recursion from y_{-1} gives $y_t = \alpha \sum_{k=0}^{t} (1-\alpha)^k x_{t-k} + (1-\alpha)^{t+1} y_{-1}$. With a zero state, $\tilde{y}_t = \alpha \sum_{k=0}^{t} (1-\alpha)^k (x_{t-k} - x_0)$. Using $\alpha \sum_{k=0}^{t} (1-\alpha)^k = 1 - (1-\alpha)^{t+1}$, adding x_0 gives the formula with $y_{-1} = x_0$. □

Starting from zero instead would bias the early outputs towards zero, so the filter seeds with the first sample by default.

Parameterizations. Two helpers convert the usual parameterizations to α:

- `alpha_from_span`$(N) = 2/(N+1)$. The mean age of the EWMA weights is $\sum_k k\,\alpha(1-\alpha)^k = (1-\alpha)/\alpha$, and that of an N-point moving average is $(N-1)/2$. Equating the two gives $\alpha = 2/(N+1)$.
- `alpha_from_halflife`$(h) = 1 - 2^{-1/h}$. This solves $(1-\alpha)^h = \frac{1}{2}$: the weight h steps back is half the newest weight.

Code 2: EWMA on iir1's one-pole section, with first-sample seeding
`src/signal/EWMA.cpp:16-36`

```
struct EWMA::Impl {
    Iir::Custom::OnePole section;
    Iir::DirectFormI state;
    bool seed = true;
    bool started = false;
    double offset = 0.0;

    double process(double x) {
        if (!started) {
            offset = seed ? x : 0.0;
            started = true;
        }
        return section.filter(x - offset, state) + offset;
    }

    void reset() {
        state.reset();
        started = false;
        offset = 0.0;
    }
};
```

Code

class and parameters — `include/nabia/signal/EWMA.h:23-43`
span and half-life conversions — `src/signal/EWMA.cpp:72-79`
simple moving average helper (rolling sum, $O(n)$) — `src/signal/smoothing.cpp:7-28`

4.2 Butterworth band-pass

A causal Butterworth band-pass isolates trend cycles between two frequencies $0 < f_\ell < f_h < f_s/2$. It is built in three steps.

1. **Analog prototype.** The order-N low-pass prototype is maximally flat:

$$|H(j\Omega)|^2 = \frac{1}{1+\Omega^{2N}},$$

   with poles $s_k = e^{j\pi(2k+N-1)/(2N)}$, $k = 1, \dots, N$, on the left half of the unit circle.

2. **Low-pass to band-pass.** The substitution

$$s \mapsto \frac{s^2+\Omega_0^2}{sB}$$

   maps it to a band-pass with $2N$ poles, centred on Ω_0 with bandwidth B.

3. **Bilinear transform.** The map $s = 2f_s\frac{1-z^{-1}}{1+z^{-1}}$, with the band edges prewarped by $\Omega = 2f_s\tan(\pi f/f_s)$, gives the digital filter. It is run as a cascade of second-order sections, which is numerically stable at high orders.

iir1 takes the band as a centre and a width, so the edges map to centre $(f_\ell + f_h)/2$ and width $f_h - f_\ell$. The filter then has $|H| = 1/\sqrt{2}$ ($-3\,\mathrm{dB}$) at both edges and unit gain at the centre. The tests check exactly these properties, and strong attenuation outside the band. Being causal, the filter delays the signal: the group delay grows with the order and with narrower bands.

Code

parameters (order, sample rate, edges) — `include/nabia/signal/ButterworthBandPass.h:11-16`
design: edges to centre and width — `src/signal/ButterworthBandPass.cpp:19-22`

5 Forecasting

Forecasters implement `IForecaster<P>`:

- `fit(y)` estimates the model on a series;
- `forecast(H)` returns the mean $\hat{y}_{T+h}$ and the predictive variance v_h for $h = 1..H$;
- Gaussian prediction intervals follow as $\hat{y} \pm z_{(1+c)/2}\sqrt{v_h}$.

The companion documents in `doc/forecast` derive each model in full. This section states what the code computes.

5.1 Simple exponential smoothing

The model is the ETS(A,N,N) state space

$$y_t = \ell_{t-1} + \varepsilon_t, \qquad \ell_t = \ell_{t-1} + \alpha\varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, \sigma^2).$$

The fit initializes $\ell = y_1$ and runs the error-correction form $\ell \leftarrow \ell + \alpha(y_t - \ell)$, accumulating $\mathrm{SSE} = \sum_t (y_t - \ell_{t-1})^2$. It then estimates $\hat{\sigma}^2 = \mathrm{SSE}/T$.

Proposition 5.1. *The h-step forecast is $\hat{y}_{T+h} = \ell_T$, with variance $\sigma^2(1 + (h-1)\alpha^2)$.*

Proof. Iterating the state equation gives $\ell_{T+h-1} = \ell_T + \alpha \sum_{j=1}^{h-1} \varepsilon_{T+j}$, so $y_{T+h} = \ell_T + \alpha \sum_{j=1}^{h-1} \varepsilon_{T+j} + \varepsilon_{T+h}$. The errors are independent, so the variance is $(h-1)\alpha^2\sigma^2 + \sigma^2$. □

Code 3: SES fit and forecast variance
`src/forecast/exponential_smoothing.cpp:27-45`

```
    // Initialise the level with the first observation.
    level_ = y(0);
    for (Eigen::Index t = 0; t < y.size(); ++t) {
        in_sample_(t) = level_;
        const double err = y(t) - level_;
        sse_ += err * err;
        level_ += alpha * err; // equivalent to alpha*y + (1-alpha)*level
    }
    fitted_ = true;
}

Forecast SimpleExponentialSmoothing::forecast(Eigen::Index horizon) const {
    require_fitted("SimpleExponentialSmoothing::forecast");
    require_horizon(horizon, "SimpleExponentialSmoothing::forecast");
    const double alpha = parameters_->alpha;
    const double sigma2 = sse_ / static_cast<double>(in_sample_.size());
    Forecast out{Eigen::VectorXd::Constant(horizon, level_), Eigen::VectorXd(horizon)};
    for (Eigen::Index h = 0; h < horizon; ++h) {
        out.variance(h) = sigma2 * (1.0 + static_cast<double>(h) * alpha * alpha);
```

5.2 Kalman filter

The model is the linear Gaussian state space

$$x_t = Fx_{t-1} + w_t, \quad w_t \sim \mathcal{N}(0, Q); \qquad y_t = Hx_t + v_t, \quad v_t \sim \mathcal{N}(0, R),$$

with prior $x_0 \sim \mathcal{N}(\hat{x}_0, P_0)$, where the prior describes the first state. Each step first predicts:

$$\hat{x}_{t|t-1} = F\hat{x}_{t-1|t-1}, \qquad P_{t|t-1} = FP_{t-1|t-1}F^\top + Q.$$

Then it computes the innovation and its variance:

$$e_t = y_t - H\hat{x}_{t|t-1}, \qquad S_t = HP_{t|t-1}H^\top + R.$$

Finally it corrects, with gain $K_t = P_{t|t-1}H^\top / S_t$:

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t e_t, \qquad P_{t|t} = (I - K_t H)P_{t|t-1}(I - K_t H)^\top + R\,K_t K_t^\top.$$

- **Joseph form.** The covariance update above is the posterior covariance for *any* gain. It is symmetric and positive semi-definite by construction, so it stays valid under rounding, unlike the short form $(I - KH)P$.
- **Missing observations.** Non-finite y_t skip the correction.
- **Log-likelihood.** $\ell = -\frac{1}{2}\sum_t \left(\log 2\pi + \log S_t + e_t^2/S_t\right)$, summed over the observed points.
- **Forecasts.** They iterate the prediction step: mean Hx, variance $HPH^\top + R$.
- **Smoothing.** The Rauch–Tung–Striebel smoother uses the gain $C_t = P_{t|t}F^\top P_{t+1|t}^{-1}$ (computed by an $LDL^\top$ solve, not an inverse) and $\hat{x}_t^s = \hat{x}_{t|t} + C_t(\hat{x}_{t+1}^s - \hat{x}_{t+1|t})$.

Two ready-made models are provided:

- *local level*: $F = H = 1$, so the level is a random walk observed with noise;
- *local linear trend*: $F = \left(\begin{smallmatrix} 1 & 1 \\ 0 & 1 \end{smallmatrix}\right)$, $H = (1, 0)$, $Q = \mathrm{diag}(q_\ell, q_s)$, so level and slope both drift.

Both use a diffuse prior ($P_0 = 10^7 I$). The local linear trend's slope state is a direct estimate of the trend.

Code 4: Kalman predict and Joseph-form correction
`src/forecast/KalmanFilter.cpp:91-117`

```
if (t > 0) {  // predict; the prior already describes x[0]
    x = F * x;
    P = F * P * F.transpose() + p.process_noise;
}
const auto ts = static_cast<std::size_t>(t);
predicted_states_.col(t) = x;
predicted_covariances_[ts] = P;

const double y_hat = H.dot(x);
const VectorXd PHt = P * H.transpose();
const double S = H.dot(PHt) + p.observation_noise;
one_step_(t) = y_hat;
innovation_variances_(t) = S;

if (std::isfinite(y(t))) {  // correct
    const double e = y(t) - y_hat;
    const VectorXd K = PHt / S;
    x += K * e;
    const MatrixXd A = I - K * H;
    P = A * P * A.transpose() + p.observation_noise * K * K.transpose();
    innovations_(t) = e;
    log_likelihood_ -= 0.5 * (log_2pi + std::log(S) + e * e / S);
} else {
    innovations_(t) = std::numeric_limits<double>::quiet_NaN();
}
filtered_states_.col(t) = x;
filtered_covariances_[ts] = P;
```

Code

parameters F, H, Q, R, x_0, P_0 — `include/nabia/forecast/KalmanFilter.h:16-34`
local level and local linear trend — `src/forecast/KalmanFilter.cpp:25-48`
forecast — `src/forecast/KalmanFilter.cpp:122-136`
RTS smoother — `src/forecast/KalmanFilter.cpp:168-182`

5.3 Gaussian process regression

The series is regressed on time $t = 0, \ldots, T-1$ in three steps.

1. **Standardization.** The observations are standardized, $z = (y - \bar{y})/s$, where s is the sample standard deviation when normalizing.

2. **Model.** $z \sim \mathcal{N}(0, K)$ with $K = K_f + \sigma_n^2 I$ and $(K_f)_{ij} = k(t_i, t_j)$. The available kernels, with $r = t - t'$:

$$
\begin{aligned}
k_{\mathrm{SE}} &= \sigma^2 \exp(-r^2/2\ell^2), \\
k_{\mathrm{M52}} &= \sigma^2 (1 + u + u^2/3) e^{-u}, \quad u = \sqrt{5}|r|/\ell, \\
k_{\mathrm{per}} &= \sigma^2 \exp(-2\sin^2(\pi r/p)/\ell^2).
\end{aligned}
$$

   Sums and products of kernels are kernels too.

3. **Hyperparameters.** With $\alpha = K^{-1} z$ (from a Cholesky factorization $K = LL^\top$), the log marginal likelihood is

$$
\log p(z) = -\tfrac{1}{2} z^\top \alpha - \textstyle\sum_i \log L_{ii} - \tfrac{T}{2} \log 2\pi.
$$

The hyperparameters θ (logs of variances, length scales, periods and, optionally, the noise) maximize it, using L-BFGS with the analytic gradient

$$\frac{\partial(-\log p)}{\partial \theta_j} = -\tfrac{1}{2}\operatorname{tr}\Big(\big(\alpha\alpha^\top - K^{-1}\big)\frac{\partial K}{\partial \theta_j}\Big).$$

Working in logs keeps the parameters positive. Values with $|\theta_j| > 20$, or a K that is not positive definite, are returned as $+\infty$, which the line search rejects. The noise has a floor (`min_noise_variance`).

Prediction. At new times t_*, with $v = L^{-1}k_*$:

$$\text{mean} = \bar{y} + s\,k_*^\top \alpha, \qquad \text{variance} = s^2\big(k(t_*,t_*) - v^\top v + \sigma_n^2\big).$$

The reported likelihood is converted back to the units of y by subtracting $T \log s$.

Code 5: Gaussian process: negative log-likelihood gradient
`src/forecast/GaussianProcess.cpp:100-111`

```
// d(-log p)/d theta_j = -1/2 tr((alpha alpha' - K^-1) dK/d theta_j)
const MatrixXd W = f->alpha * f->alpha.transpose() - f->llt.solve(MatrixXd::Identity(T, T));
grad.setZero();
for (Index j = 0; j < T; ++j) {
    for (Index i = 0; i < T; ++i) {
        grad.head(nk) -= 0.5 * W(i, j) * work->gradient(times_(i), times_(j));
    }
}
if (p.optimize_noise) {
    grad(nk) = -0.5 * noise * W.trace();  // dK/dlog(noise) = noise I
}
return -f->log_ml;
```

Code

Cholesky and log marginal likelihood — `src/forecast/GaussianProcess.cpp:31-42`
fit with hyperparameter optimization — `src/forecast/GaussianProcess.cpp:56-133`
predictive mean and variance — `src/forecast/GaussianProcess.cpp:135-147`
kernels and their log-parameter gradients — `src/forecast/Kernels.cpp:41-226`

6 Optimization

Optimizers implement `IOptimizer<P,R>`: `optimize(initial_guess)` returns the solution, and `result()` returns a solver-specific result object (status, iterations, residuals). Linear constraints are expressed in matrix form: $A_{\text{eq}}x = b_{\text{eq}}$ and $A_{\text{in}}x \le b_{\text{in}}$. Box bounds $l \le x \le u$ convert to inequality rows.

Code

optimizer interface — `include/nabia/optimization/IOptimizer.h:16-39`
linear constraints and their conversion — `include/nabia/optimization/convex/Constraints.h:22-47`

6.1 L-BFGS

L-BFGS minimizes a smooth f with search direction $d = -H_k g_k$. Here H_k is the BFGS inverse-Hessian approximation built from the last m pairs $s_j = x_{j+1} - x_j$, $y_j = g_{j+1} - g_j$, with $\rho_j = 1/s_j^\top y_j$. It is applied in $O(mn)$ by the two-loop recursion:

1. $q \leftarrow g$. For j from newest to oldest: $a_j = \rho_j s_j^\top q$, then $q \leftarrow q - a_j y_j$.
2. $r \leftarrow \gamma q$, with $\gamma = s^\top y / y^\top y$ from the newest pair. On the first iteration $\gamma = 1/\max(1, \|g\|)$.

3. For j from oldest to newest: $\beta = \rho_j y_j^\top r$, then $r \leftarrow r + (a_j - \beta)s_j$. Finally $d = -r$.

Safeguards keep the method well defined:

- A pair is stored only if $s^\top y > 10^{-12}\|s\|\|y\|$ (the curvature condition), which keeps H positive definite.
- If d is not a descent direction, the memory is cleared and the method restarts from steepest descent.
- Iteration stops when $\|g\|_\infty \le \varepsilon$.

Line search. Backtracking from $t = 1$ halves t until the step is accepted. A non-finite f counts as rejection, which is how barrier and domain constraints are enforced. A step is accepted under either condition:

Armijo $f(x + td) \le f(x) + c_1 t\, g^\top d$, with $c_1 = 10^{-4}$.

Approximate Wolfe (Hager–Zhang), when f is flat to within rounding: $f(x + td) \le f(x) + \epsilon_f |f(x)|$ and $\sigma\, g^\top d \le g(x + td)^\top d \le (2c_1 - 1)\, g^\top d$, with $\epsilon_f = 10^{-10}$ and $\sigma = 0.9$.

Near a minimum the decrease Armijo requires is smaller than the rounding error in f, while directional derivatives remain accurate. The second test lets the method keep converging there. On 500-asset risk budgeting it took the attainable budget accuracy from about $3 \cdot 10^{-8}$ to 10^{-8}.

Code 6: Two-loop recursion
`src/optimization/convex/LBFGS.cpp:63-77`

```cpp
// Two-loop recursion: d = -H g.
VectorXd q = g;
std::vector<double> alpha(s_hist.size());
for (std::size_t j = s_hist.size(); j-- > 0;) {
    alpha[j] = rho_hist[j] * s_hist[j].dot(q);
    q -= alpha[j] * y_hist[j];
}
const double gamma = s_hist.empty() ? 1.0 / std::max(1.0, g.norm())
                                    : s_hist.back().dot(y_hist.back()) / y_hist.back().squaredNorm();
VectorXd d = gamma * q;
for (std::size_t j = 0; j < s_hist.size(); ++j) {
    const double beta = rho_hist[j] * y_hist[j].dot(d);
    d += (alpha[j] - beta) * s_hist[j];
}
d = -d;
```

Code 7: Line search: Armijo or approximate Wolfe
`src/optimization/convex/LBFGS.cpp:88-116`

```cpp
double t = 1.0;
VectorXd x_new;
VectorXd g_new(x.size());
double f_new = fx;
bool accepted = false;
for (int ls = 0; ls < p.max_line_search; ++ls) {
    x_new = x + t * d;
    f_new = value_and_gradient_(x_new, g_new);
    ++result_.evaluations;
    if (!std::isfinite(f_new)) {
        t *= p.backtrack;
        continue;
    }
    // Armijo sufficient decrease, or Hager-Zhang's approximate Wolfe
    // conditions when f is flat to within rounding: near the minimum
    // the decrease in f drops below machine precision, but the
    // directional derivative is still accurate.
    const double new_slope = g_new.dot(d);
    const bool armijo = f_new <= fx + p.armijo_c1 * t * slope;
```

```
    const bool approx_wolfe = f_new <= fx + p.flat_tolerance * std::abs(fx) &&
                              new_slope <= (2.0 * p.armijo_c1 - 1.0) * slope &&
                              new_slope >= p.wolfe_c2 * slope;
    if (armijo || approx_wolfe) {
        accepted = true;
        break;
    }
    t *= p.backtrack;
}
if (!accepted) {
```

Code

parameters and defaults — `include/nabia/optimization/convex/LBFGS.h:12-25`
full algorithm — `src/optimization/convex/LBFGS.cpp:30-148`

6.2 Augmented Lagrangian

For $\min f(x)$ subject to $h(x) = A_{\text{eq}}x - b_{\text{eq}} = 0$ and $g(x) = A_{\text{in}}x - b_{\text{in}} \le 0$, the Powell–Hestenes–Rockafellar augmented Lagrangian is

$$L_\rho(x;\lambda,\mu) = f(x) + \lambda^\top h + \frac{\rho}{2}\|h\|^2 + \frac{1}{2\rho}\sum_i \Big(\max(0,\mu_i + \rho g_i)^2 - \mu_i^2\Big),$$

with gradient $\nabla f + A_{\text{eq}}^\top(\lambda + \rho h) + A_{\text{in}}^\top \max(0, \mu + \rho g)$. Each outer iteration does four things:

1. **Minimize** L_ρ with L-BFGS, warm-started at the previous iterate. The inner tolerance starts at $\max(\varepsilon, 0.1)$ and shrinks tenfold per outer iteration down to ε.
2. **Measure** the violation $v = \max\left(\|h\|_\infty, \|\max(g, -\mu/\rho)\|_\infty\right)$. The second term is zero if and only if $g \le 0$ and $\mu_i g_i = 0$, so it also measures complementarity.
3. **Update the multipliers**: $\lambda \leftarrow \lambda + \rho h$ and $\mu \leftarrow \max(0, \mu + \rho g)$. After this update, $\nabla_x L_\rho = \nabla f + A_{\text{eq}}^\top\lambda + A_{\text{in}}^\top\mu$, so the inner gradient norm is the KKT stationarity.
4. **Test and adapt.** Stop when $v \le \varepsilon$ and stationarity $\le \varepsilon$. Otherwise, if $v > 0.25\, v_{\text{prev}}$, set $\rho \leftarrow \min(10\rho, 10^{10})$ (starting from $\rho_0 = 10$).

Code 8: Augmented Lagrangian value and gradient
`src/optimization/convex/AugmentedLagrangian.cpp:58-73`

```
// L_rho(x) = f + lambda'h + rho/2 |h|^2 + 1/(2 rho) sum(max(0, mu + rho g)^2 - mu^2)
// grad     = grad f + A_eq'(lambda + rho h) + A_ineq' max(0, mu + rho g)
const LBFGS::ValueAndGradient augmented = [&](const VectorXd& z, VectorXd& grad) {
    double value = objective_(z);
    grad = gradient_(z);
    if (has_eq) {
        const VectorXd h = lf.A_eq * z - lf.b_eq;
        value += lambda.dot(h) + 0.5 * rho * h.squaredNorm();
        grad.noalias() += lf.A_eq.transpose() * (lambda + rho * h);
    }
    if (has_ineq) {
        const VectorXd shifted = (mu + rho * (lf.A_ineq * z - lf.b_ineq)).cwiseMax(0.0);
        value += (shifted.squaredNorm() - mu.squaredNorm()) / (2.0 * rho);
        grad.noalias() += lf.A_ineq.transpose() * shifted;
    }
    return value;
```

Code 9: Multiplier and penalty updates
`src/optimization/convex/AugmentedLagrangian.cpp:93-119`

```
// Violation measured before the multiplier update; for inequalities
// max(g, -mu/rho) is zero iff g <= 0 and mu * g = 0 (complementarity).
double violation = 0.0;
```

```cpp
if (has_eq) {
    const VectorXd h = lf.A_eq * x - lf.b_eq;
    violation = h.lpNorm<Eigen::Infinity>();
    lambda += rho * h;
}
if (has_ineq) {
    const VectorXd g = lf.A_ineq * x - lf.b_ineq;
    violation = std::max(violation, g.cwiseMax(-mu / rho).lpNorm<Eigen::Infinity>());
    mu = (mu + rho * g).cwiseMax(0.0);
}

// After the update, grad L_rho(x) = grad f + A_eq' lambda + A_ineq' mu.
result_.stationarity = inner.gradient_norm;
result_.constraint_violation = violation;
if (violation <= p.tolerance && inner.gradient_norm <= p.tolerance) {
    result_.converged = true;
    break;
}

if (violation > options_.required_decrease * previous_violation) {
    rho = std::min(rho * options_.penalty_growth, options_.max_penalty);
}
previous_violation = violation;
inner_tol = std::max(p.tolerance, 0.1 * inner_tol);
```

Code

options and defaults — `include/nabia/optimization/convex/AugmentedLagrangian.h:15-23`
full algorithm — `src/optimization/convex/AugmentedLagrangian.cpp:33-128`

6.3 Sequential quadratic programming

At x_k, SQP solves the quadratic subproblem

$$\min_d \; g_k^\top d + \tfrac{1}{2} d^\top B_k d \quad \text{s.t.} \quad A_{\text{eq}} d = b_{\text{eq}} - A_{\text{eq}} x_k, \;\; A_{\text{in}} d \le b_{\text{in}} - A_{\text{in}} x_k.$$

It uses the augmented Lagrangian of §6.2 at tolerance 10^{-10}, or a Cholesky solve $d = -B^{-1} g$ when there are no constraints.

Convergence is declared when three KKT residuals at x_k, computed with the QP multipliers, are all below ε:

- stationarity $\|g + A_{\text{eq}}^\top \lambda + A_{\text{in}}^\top \mu\|_\infty$;
- constraint violation;
- complementarity $\max_i |\mu_i g_i|$.

Merit function $\phi(x) = f(x) + \nu V_1(x)$, with $V_1 = \|h\|_1 + \sum_i \max(g_i, 0)$. The penalty is kept at $\nu \ge \|(\lambda, \mu)\|_\infty + 10^{-3}$. Because the constraints are linear and d satisfies the linearized constraints, $V_1(x + td) \le (1 - t) V_1(x)$. The directional derivative is therefore bounded by $g^\top d - \nu V_1(x) < 0$, so d descends ϕ. Backtracking applies Armijo to ϕ with this slope.

Restart If the line search fails, B resets to the identity. A failure from $B = I$ ends the iteration.

Proposition 6.1 (Damped BFGS, Powell)**.** *Let $r = \theta y + (1 - \theta) Bs$, with $\theta = 1$ if $s^\top y \ge 0.2\, s^\top Bs$ and $\theta = 0.8\, s^\top Bs / (s^\top Bs - s^\top y)$ otherwise. The update*

$$B^+ = B - \frac{Bss^\top B}{s^\top Bs} + \frac{rr^\top}{s^\top r}$$

keeps B positive definite.

Proof. If $\theta < 1$, then $s^\top r = s^\top Bs - \theta(s^\top Bs - s^\top y) = 0.2\, s^\top Bs > 0$. If $\theta = 1$, then $s^\top r = s^\top y \geq 0.2\, s^\top Bs > 0$. The BFGS update with a pair satisfying $s^\top r > 0$ preserves positive definiteness. □

After the first step with $s^\top y > 0$, B is scaled to $(y^\top y / s^\top y) I$ before the update, which matches the curvature along s.

Code 10: Damped BFGS update
`src/optimization/convex/SQP.cpp:41-53`

```
void damped_bfgs_update(MatrixXd& B, const VectorXd& s, const VectorXd& y) {
    const VectorXd Bs = B * s;
    const double sBs = s.dot(Bs);
    if (!(sBs > 0.0)) {
        return;
    }
    const double sy = s.dot(y);
    const double theta = sy >= 0.2 * sBs ? 1.0 : 0.8 * sBs / (sBs - sy);
    const VectorXd r = theta * y + (1.0 - theta) * Bs;
    B.noalias() -= Bs * Bs.transpose() / sBs;
    B.noalias() += r * r.transpose() / s.dot(r);
    B = 0.5 * (B + B.transpose());  // remove round-off asymmetry
}
```

Code 11: ℓ_1 merit penalty and line search
`src/optimization/convex/SQP.cpp:156-187`

```
// l1 merit function phi = f + nu * V1; nu must exceed the multipliers
// for d to be a descent direction of phi.
double max_multiplier = 0.0;
if (lambda.size() > 0) {
    max_multiplier = lambda.lpNorm<Eigen::Infinity>();
}
if (mu.size() > 0) {
    max_multiplier = std::max(max_multiplier, mu.lpNorm<Eigen::Infinity>());
}
nu = std::max(nu, max_multiplier + options_.merit_penalty_margin);
const double v1 = l1_violation(x);
const double merit = fx + nu * v1;
// Directional derivative of phi along d (the constraints are linear and
// d satisfies them, so V1(x + t d) <= (1 - t) V1(x)).
double slope = gx.dot(d) - nu * v1;
if (!(slope < 0.0)) {
    slope = -1e-16 * std::max(1.0, std::abs(merit));  // d ~ 0: accept the first decrease
}

double t = 1.0;
VectorXd x_new;
double f_new = fx;
bool accepted = false;
for (int ls = 0; ls < options_.max_line_search; ++ls) {
    x_new = x + t * d;
    f_new = objective_(x_new);
    ++result_.evaluations;
    if (std::isfinite(f_new) && f_new + nu * l1_violation(x_new) <= merit + options_.merit_c1 * t *
    ↪ slope) {
        accepted = true;
        break;
    }
    t *= options_.backtrack;
```

Code

options and defaults — `include/nabia/optimization/convex/SQP.h:15-23`
QP subproblem through the augmented Lagrangian — `src/optimization/convex/SQP.cpp:26-38`
full algorithm — `src/optimization/convex/SQP.cpp:66-218`

6.4 Nelder–Mead with box bounds

Nelder–Mead keeps a simplex of $n+1$ vertices sorted by value, $f(x_0) \le \cdots \le f(x_n)$, with c the centroid of the best n. Each iteration:

1. **Reflect**: $x_r = c + \alpha(c - x_n)$.
2. If $f_r < f_0$, **expand** to $x_e = c + \gamma(x_r - c)$ and keep the better of x_e and x_r.
3. Else, if $f_r < f_{n-1}$, accept x_r.
4. Else **contract**:
   - outside, $x_c = c + \rho(x_r - c)$, if $f_r < f_n$ (accepted if $f_c < f_r$);
   - inside, $x_c = c + \rho(x_n - c)$, otherwise (accepted if $f_c < f_n$).
5. Else **shrink** every vertex towards the best: $x_j \leftarrow x_0 + \sigma(x_j - x_0)$.

The *adaptive* coefficients of Gao and Han, $\alpha = 1$, $\gamma = 1 + 2/n$, $\rho = \frac{3}{4} - \frac{1}{2n}$, $\sigma = 1 - \frac{1}{n}$, avoid the stagnation of the standard values $(1, 2, \frac{1}{2}, \frac{1}{2})$ in higher dimensions.

Bounds are enforced by projecting every trial point onto the box. The box is convex, so a shrink stays inside it.

Termination. The iteration stops when both conditions hold:

$$f_n - f_0 \le \tau_f(1 + |f_0|) \quad \text{and} \quad \max_j \|x_j - x_0\|_\infty \le \tau_x(1 + \|x_0\|_\infty).$$

A NaN objective counts as $+\infty$.

Code 12: One Nelder–Mead iteration
`src/optimization/nongradopt/NelderMead.cpp:119-154`

```
const VectorXd centroid = simplex.leftCols(n).rowwise().mean();
const VectorXd worst = simplex.col(n);
const VectorXd xr = box.project(centroid + alpha * (centroid - worst));
const double fr = f(xr);

if (fr < values(0)) {
    const VectorXd xe = box.project(centroid + gamma * (xr - centroid));
    const double fe = f(xe);
    if (fe < fr) {
        simplex.col(n) = xe;
        values(n) = fe;
    } else {
        simplex.col(n) = xr;
        values(n) = fr;
    }
    continue;
}
if (fr < values(n - 1)) {
    simplex.col(n) = xr;
    values(n) = fr;
    continue;
}
// Contraction: outside if the reflection beat the worst vertex, inside otherwise.
const bool outside = fr < values(n);
const VectorXd xc = outside ? VectorXd(box.project(centroid + rho * (xr - centroid)))
                            : VectorXd(box.project(centroid + rho * (worst - centroid)));
const double fc = f(xc);
if (fc < (outside ? fr : values(n))) {
    simplex.col(n) = xc;
    values(n) = fc;
    continue;
}
// Shrink towards the best vertex (stays inside the box: the box is convex).
for (Index j = 1; j <= n; ++j) {
    simplex.col(j) = simplex.col(0) + sigma * (simplex.col(j) - simplex.col(0));
    values(j) = f(simplex.col(j));
```

Code

adaptive coefficients — `src/optimization/nongradopt/NelderMead.cpp:43-46`
box projection — `src/optimization/nongradopt/box.hpp:17-23`
full algorithm, initial simplex — `src/optimization/nongradopt/NelderMead.cpp:24-162`

6.5 Differential evolution

Population. N_p vectors (default $15n$, at least 4, or 6 for rand/2), drawn uniformly in the box. An initial guess, if given, replaces member 0.

Mutation. Each generation draws one $F \sim U[F_{\min}, F_{\max}]$ (dithering), then builds a mutant per member i from distinct random members $r_1, \ldots, r_5 \neq i$:

$$\begin{aligned}
\text{rand/1:} \quad & v = x_{r_1} + F(x_{r_2} - x_{r_3}), \\
\text{best/1:} \quad & v = x_{\text{best}} + F(x_{r_1} - x_{r_2}), \\
\text{current-to-best/1:} \quad & v = x_i + F(x_{\text{best}} - x_i) + F(x_{r_1} - x_{r_2}), \\
\text{rand/2:} \quad & v = x_{r_1} + F(x_{r_2} - x_{r_3}) + F(x_{r_4} - x_{r_5}).
\end{aligned}$$

Crossover. Binomial: $u_{ij} = v_{ij}$ if $U < \text{CR}$ or $j = j_{\text{rand}}$, else x_{ij}, so at least one coordinate comes from the mutant. A coordinate outside the box bounces back to the midpoint between the parent and the violated bound.

Selection. Greedy: u replaces x_i if $f(u) \leq f(x_i)$. Ties go to the trial, which keeps the population moving on plateaus.

Termination. The run stops when the population's objective values have $\text{sd}(f) \leq \tau_a + \tau|\overline{f}|$.

Parallelism and reproducibility. All trial vectors of a generation are built sequentially from one seeded RNG *before* evaluation. Only the evaluations run in parallel, so the result is the same for every thread count. A `ParallelEvaluator<T,R>` does the evaluations:

- each input becomes a `std::packaged_task` in a queue;
- n worker threads consume the queue;
- the caller collects the results, in input order, through the tasks' futures.

Code 13: Mutation and crossover of one generation
`src/optimization/nongradopt/DifferentialEvolution.cpp:132-172`

```
const double F = p.mutation_min + (p.mutation_max - p.mutation_min) * unit(rng);

// Build all trial vectors first (sequential RNG => thread-count independent).
for (Index j = 0; j < np; ++j) {
    VectorXd mutant(n);
    switch (p.strategy) {
    case DEStrategy::Rand1: {
        const auto r = pick_distinct<3>(rng, np, j);
        mutant = population.col(r[0]) + F * (population.col(r[1]) - population.col(r[2]));
        break;
    }
    case DEStrategy::Best1: {
        const auto r = pick_distinct<2>(rng, np, j);
        mutant = population.col(best) + F * (population.col(r[0]) - population.col(r[1]));
        break;
    }
    case DEStrategy::CurrentToBest1: {
        const auto r = pick_distinct<2>(rng, np, j);
```

```
        mutant = population.col(j) + F * (population.col(best) - population.col(j)) +
                 F * (population.col(r[0]) - population.col(r[1]));
        break;
    }
    case DEStrategy::Rand2: {
        const auto r = pick_distinct<5>(rng, np, j);
        mutant = population.col(r[0]) + F * (population.col(r[1]) - population.col(r[2])) +
                 F * (population.col(r[3]) - population.col(r[4]));
        break;
    }
    }

    // Binomial crossover; coordinate j_rand always comes from the mutant.
    const Index j_rand = any_coordinate(rng);
    for (Index i = 0; i < n; ++i) {
        double v = (i == j_rand || unit(rng) < p.crossover) ? mutant(i) : population(i, j);
        // Bounce back halfway between the parent and the violated bound.
        if (v < box.lower(i)) {
            v = 0.5 * (box.lower(i) + population(i, j));
        } else if (v > box.upper(i)) {
            v = 0.5 * (box.upper(i) + population(i, j));
        }
        trials(i, j) = v;
```

Code 14: ParallelEvaluator: job queue, consumers, futures
`src/optimization/nongradopt/parallel_evaluator.hpp:33-76`

```
[[nodiscard]] std::vector<TResult> evaluate(const std::vector<T>& inputs) const {
    std::queue<std::packaged_task<TResult()>> jobs;
    std::vector<std::future<TResult>> futures;
    futures.reserve(inputs.size());
    for (const T& input : inputs) {
        std::packaged_task<TResult()> job([this, &input] { return objective_(input); });
        futures.push_back(job.get_future());
        jobs.push(std::move(job));
    }

    std::mutex mutex;  // guards jobs
    auto consume = [&] {
        for (;;) {
            std::packaged_task<TResult()> job;
            {
                const std::lock_guard lock(mutex);
                if (jobs.empty()) {
                    return;
                }
                job = std::move(jobs.front());
                jobs.pop();
            }
            job();  // stores the result, or the exception, in the job's promise
        }
    };

    // Declared after jobs and mutex: on any exit the consumers are joined
    // (and finish the queue) before what they use is destroyed.
    std::vector<std::jthread> consumers;
    if (num_threads_ == 1) {
        consume();
    } else {
        consumers.reserve(static_cast<std::size_t>(num_threads_));
        for (int t = 0; t < num_threads_; ++t) {
            consumers.emplace_back(consume);
        }
    }
    std::vector<TResult> results;
    results.reserve(inputs.size());
    for (std::future<TResult>& f : futures) {
        results.push_back(f.get());  // waits; rethrows the job's exception
    }
    return results;
}
```

Code

parameters and defaults — `include/nabia/optimization/nongradopt/DifferentialEvolution.h:24-40`
full algorithm — `src/optimization/nongradopt/DifferentialEvolution.cpp:50-193`

7 Risk measures

For portfolio weights x on assets with return covariance Σ and mean μ, write $\sigma(x) = \sqrt{x^\top \Sigma x}$ and take the loss $L = -r^\top x$ with $r \sim \mathcal{N}(\mu, \Sigma)$. Then $L \sim \mathcal{N}(-\mu^\top x, \sigma(x)^2)$. The three measures are:

$$\begin{aligned}
\text{Volatility:} \quad & R(x) = \sigma(x), && \nabla R = \Sigma x/\sigma(x), \\
\text{VaR}_\alpha: \quad & R(x) = -\mu^\top x + z_\alpha\, \sigma(x), && \nabla R = -\mu + z_\alpha \Sigma x/\sigma(x), \\
\text{ES}_\alpha: \quad & R(x) = -\mu^\top x + \frac{\varphi(z_\alpha)}{1-\alpha}\, \sigma(x), && \nabla R = -\mu + \frac{\varphi(z_\alpha)}{1-\alpha} \Sigma x/\sigma(x),
\end{aligned}$$

with $z_\alpha = \Phi^{-1}(\alpha)$.

Proposition 7.1 (Gaussian expected shortfall)**.** *If $L \sim \mathcal{N}(m, s^2)$, then* $\text{ES}_\alpha = \mathbb{E}[L \mid L \geq \text{VaR}_\alpha] = m + s\,\varphi(z_\alpha)/(1-\alpha)$.

Proof. Write $L = m + sZ$. The event is $Z \geq z_\alpha$, which has probability $1-\alpha$. Since $\varphi'(z) = -z\varphi(z)$, we get $\int_{z_\alpha}^{\infty} z\varphi(z)\, dz = \varphi(z_\alpha)$, so $\mathbb{E}[Z \mid Z \geq z_\alpha] = \varphi(z_\alpha)/(1-\alpha)$. □

Proposition 7.2 (Euler decomposition)**.** *Each measure is positively homogeneous of degree one, $R(cx) = cR(x)$ for $c > 0$. Hence $R(x) = \sum_i x_i\, \partial_i R(x)$, and the* risk contributions $\text{RC}_i = x_i \partial_i R$ *add up exactly to the total risk.*

Proof. Differentiate $R(cx) = cR(x)$ with respect to c at $c = 1$. □

At $\sigma = 0$ the gradient of σ is set to zero, which is a valid subgradient of the seminorm. The quantile Φ^{-1} uses Acklam's rational approximation (relative error below $1.2 \cdot 10^{-9}$), refined to full precision by one Halley step on $\Phi(x) - p$:

$$x \leftarrow x - \frac{u}{1 + xu/2}, \qquad u = (\Phi(x) - p)\sqrt{2\pi}\, e^{x^2/2}.$$

Code 15: Volatility gradient, the building block of every measure
`src/risk/IRiskMeasure.cpp:47-55`

```
Eigen::VectorXd IRiskMeasure::portfolio_volatility_gradient(const Eigen::VectorXd& x) const {
    check_dimension(x, "portfolio");
    Eigen::VectorXd sigma_x = covariance_matrix_ * x;
    const double sigma = std::sqrt(std::max(0.0, x.dot(sigma_x)));
    if (sigma == 0.0) {
        return Eigen::VectorXd::Zero(dim());
    }
    return sigma_x / sigma;
}
```

Code 16: Gaussian VaR and its gradient
`src/risk/measures/ValueAtRisk.cpp:27-32`

```
double ValueAtRisk::compute_risk(const Eigen::VectorXd& x) const {
    return -expected_returns_.dot(x) + z_ * portfolio_volatility(x);
}

Eigen::VectorXd ValueAtRisk::compute_marginal_risk_contributions(const Eigen::VectorXd& x) const {
    return -expected_returns_ + z_ * portfolio_volatility_gradient(x);
```

Code

risk measure interface — `include/nabia/risk/IRiskMeasure.h:18-43`
risk contributions $x \odot \nabla R$ — `src/risk/IRiskMeasure.cpp:31-33`
volatility — `src/risk/measures/Volatility.cpp:7-12`
expected shortfall (constant $\varphi(z_\alpha)/(1-\alpha)$) — `src/risk/measures/ExpectedShortfall.cpp:12-33`
normal quantile: Halley refinement — `src/stats/normal.cpp:50-53`

8 Risk budgeting

Given budgets $b_i > 0$ with $\sum_i b_i = 1$, the *risk budgeting portfolio* is the long-only x with $\mathrm{RC}_i(x)/R(x) = b_i$ for all i. Equal budgets give the equal risk contribution portfolio.

Theorem 8.1. *Let R be convex, positively homogeneous of degree one, differentiable on $y > 0$, and positive on $\{y \geq 0,\ y \neq 0\}$. Then*

$$f(y) = R(y) - \sum_i b_i \log y_i$$

has a unique minimizer $y^* > 0$, and $x^* = y^*/\sum_i y_i^*$ is the unique long-only portfolio with $\mathrm{RC}_i(x^*)/R(x^*) = b_i$. Moreover $R(y^*) = 1$.

Proof. Existence. R is continuous and positive on the compact simplex $\Delta = \{y \geq 0, \sum y_i = 1\}$, so $c = \min_\Delta R > 0$. By homogeneity, $R(y) \geq c \sum_i y_i$ for $y \geq 0$. Hence $f(y) \geq \sum_i (c y_i - b_i \log y_i)$. Each term tends to $+\infty$ as $y_i \to 0^+$ or $y_i \to \infty$, so f is coercive on $y > 0$ and attains its minimum.

Uniqueness. $-\sum b_i \log y_i$ is strictly convex and R is convex, so f is strictly convex.

Characterization. At the minimizer, $\partial_i f = \partial_i R(y^*) - b_i/y_i^* = 0$, so $\mathrm{RC}_i(y^*) = y_i^* \partial_i R(y^*) = b_i$. Summing and using Euler gives $R(y^*) = \sum_i b_i = 1$. Let $S = \sum_i y_i^*$. Because $\partial_i R$ is homogeneous of degree zero, $x^* = y^*/S$ has $\mathrm{RC}_i(x^*) = b_i/S$ and $R(x^*) = 1/S$, so $\mathrm{RC}_i/R = b_i$.

Conversely, if x satisfies the budget condition, then $y = x/R(x)$ satisfies $\partial_i f(y) = 0$ and is therefore the unique minimizer. □

For VaR and ES, the condition "$R > 0$ on long-only portfolios" needs $z\,\sigma(x) > \mu^\top x$. The solver checks it at the start and throws otherwise.

Algorithm. The steps follow `src/risk/RiskBudgeting.cpp:45-111`.

1. **Normalize** the budgets. Empty budgets mean equal risk.
2. **Start** at $y_i^0 \propto b_i/\sigma_i$, scaled so that $R(y^0) = 1$. That scale is the magnitude of the solution by the theorem. The start is the inverse-volatility portfolio for equal budgets, which is exact for uncorrelated assets.
3. **Minimize** f with L-BFGS (memory 20). Points with any $y_i \leq 0$ return $+\infty$, so the log barrier needs no explicit constraint.
4. **Match the tolerance to the budget error.** L-BFGS stops on $\|\nabla f\|_\infty$, but the target is the budget error $e = \max_i |\mathrm{RC}_i/R - b_i|$. Near the solution $R \approx 1$ and $\mathrm{RC}_i - b_i = y_i \nabla_i f$, so $e \lesssim \max_i y_i \, \|\nabla f\|_\infty$. The inner tolerance therefore starts at $\tau/\max_i y_i^0$. If $e > \tau$ afterwards, L-BFGS warm-restarts with tolerance $\tau_{\text{in}} \cdot \frac{1}{2}\tau/e$, at most five times.
5. **Return** $x = y/\sum y$, with risk, contributions, budget error and a convergence flag.

Code 17: Objective and gradient, for any risk measure
`src/risk/RiskBudgeting.cpp:65-73`

```cpp
// f(y) = R(y) - b' log(y),  grad f(y) = dR/dy - b ./ y,  y_i grad_i = RC_i(y) - b_i.
// Points with some y_i <= 0 get +inf, which the line search rejects.
const LBFGS::ValueAndGradient objective = [&](const VectorXd& y, VectorXd& grad) {
    if ((y.array() <= 0.0).any()) {
        grad.setZero();
        return std::numeric_limits<double>::infinity();
    }
    grad = measure.compute_marginal_risk_contributions(y) - b.cwiseQuotient(y);
    return measure.compute_risk(y) - b.dot(VectorXd(y.array().log()));
```

Code 18: Tolerance matching on the budget error
`src/risk/RiskBudgeting.cpp:79-99`

```cpp
// L-BFGS stops on ||grad||_inf, but the budget error is ~ ||y .* grad||_inf,
// so start from tolerance / max(y) and, if needed, warm-restart with a
// tighter tolerance until the budget error itself is below `tolerance`.
VectorXd y = y0;
double inner_tol = params.tolerance / y0.maxCoeff();
int iterations = 0;
constexpr int max_restarts = 5;
for (int restart = 0; restart <= max_restarts && iterations < params.max_iter; ++restart) {
    LBFGSParams lbfgs_params{n};
    lbfgs_params.tolerance = inner_tol;
    lbfgs_params.max_iter = params.max_iter - iterations;
    lbfgs_params.memory = params.memory;
    LBFGS solver(std::make_unique<LBFGSParams>(lbfgs_params), objective);
    y = solver.optimize(y);
    iterations += solver.result().iterations;

    const double err = budget_error(y);
    if (err <= params.tolerance || solver.result().status == LBFGSStatus::LineSearchFailed) {
        break;
    }
    inner_tol *= 0.5 * params.tolerance / err;
```

Code

budget normalization — `src/risk/RiskBudgeting.cpp:19-30`
starting point — `src/risk/RiskBudgeting.cpp:34-41`
`RiskBudgeting<Measure>` wrapper — `include/nabia/risk/RiskBudgeting.h:51-69`

9 The walk-forward portfolio engine

`RiskBudgetingPortfolio` backtests a long-only risk budgeting portfolio from a $T \times N$ matrix of simple returns, where row t holds the returns over $(t-1, t]$. Table 1 lists its parameters, which are also the JSON keys of the CLI and the Python API.

Code

parameter struct — `include/nabia/strategy/RiskBudgetingPortfolio.h:31-57`
validation — `src/strategy/RiskBudgetingPortfolio.cpp:110-136`
the engine — `src/strategy/RiskBudgetingPortfolio.cpp:138-267`

9.1 Timing and absence of look-ahead

Rebalances happen at the close of rows $t_k = W - 1 + kf$ for $k = 0, \ldots, \lfloor (T-W)/f \rfloor$. At rebalance t the engine:

1. estimates moments from rows $t - W + 1, \ldots, t$, all known at the close of t;
2. solves for the risk budgeting weights and scales them to the volatility target;
3. trades from the drifted weights to the targets.

Table 1: Portfolio parameters: engine defaults and the defaults of the AI configuration.

Key	Engine	AI	Meaning
`covariance_window`	60	120	rows W per covariance estimate
`covariance_estimator`	sample	exponential	equal or exponential weights
`ewma_halflife`	20	40	half-life h in rows (exponential)
`budgets`	equal	set by policy	risk budgets per asset
`risk_measure`	volatility	volatility	volatility, `value_at_risk`, `expected_shortfall`
`confidence_level`	0.99	–	VaR/ES level in $(0.5, 1)$
`use_expected_returns`	false	–	VaR/ES: window mean instead of 0
`target_volatility`	none	0.10	annualized target σ^*
`max_leverage`	2.0	3.0	cap $L_{\max}$ on $\sum_i w_i$
`rebalance_frequency`	1	21	trade every k rows
`transaction_cost`	0	0.0005	cost κ per unit traded (5 bp)
`drift_weights`	true	true	weights drift between trades
`initial_capital`	1	10^6	starting equity
`periods_per_year`	252	252	P, for annualization
`solver_tolerance`	10^{-8}	–	budget error target τ
`solver_max_iter`	5000	–	L-BFGS iteration budget
`num_threads`	0 (all)	–	threads for the solves

The new weights earn the returns of rows $t+1$ onwards. A test confirms there is no look-ahead: overwriting the data from row 200 on leaves every weight up to row 200 bit-identical.

9.2 Covariance estimation

For the window's returns $r_1, \dots, r_W$ (oldest first) and weights $w_k \geq 0$ with $\sum w_k = 1$:

$$\hat{\mu} = \sum_k w_k r_k, \qquad \hat{\Sigma} = \frac{\sum_k w_k (r_k - \hat{\mu})(r_k - \hat{\mu})^\top}{1 - \sum_k w_k^2}.$$

The weights are either equal ($w_k = 1/W$) or exponential: $w_k \propto \lambda^{W-k}$ with $\lambda = 2^{-1/h}$, so the newest return has age 0 and weight 1 before normalization.

Proposition 9.1. *For i.i.d. returns with covariance Σ, $\mathbb{E}\hat{\Sigma} = \Sigma$. With equal weights the denominator is $(W-1)/W$, which recovers the usual $1/(W-1)$ estimator.*

Proof. Write $r_k - \hat{\mu} = \sum_j (\delta_{kj} - w_j)(r_j - \mu)$. By independence, $\mathbb{E}[(r_k - \hat{\mu})(r_k - \hat{\mu})^\top] = \sum_j (\delta_{kj} - w_j)^2 \Sigma = (1 - 2w_k + \sum_j w_j^2)\Sigma$. Weighting by w_k and summing gives $(1 - \sum_j w_j^2)\Sigma$. □

The quantity $1/\sum_k w_k^2$ is the effective number of observations.

Code 19: Window moments
`src/strategy/RiskBudgetingPortfolio.cpp:35-53`

```
WindowMoments estimate(const RiskBudgetingPortfolioParams& p, const MatrixXd& window) {
    const Index W = window.rows();
    VectorXd w = VectorXd::Constant(W, 1.0 / static_cast<double>(W));
    if (p.covariance_estimator == CovarianceEstimator::Exponential) {
        // Weight lambda^age with lambda = 2^(-1 / halflife); the newest row has age 0.
        const double lambda = std::exp2(-1.0 / p.ewma_halflife);
        for (Index i = 0; i < W; ++i) {
            w(i) = std::pow(lambda, static_cast<double>(W - 1 - i));
        }
        w /= w.sum();
    }
    WindowMoments m;
    m.mean = window.transpose() * w;
    const MatrixXd centred = window.rowwise() - m.mean.transpose();
    // Unbiased weighted covariance: divide by 1 - sum(w^2) (= (W - 1) / W for equal weights).
```

```
    m.covariance = centred.transpose() * w.asDiagonal() * centred / (1.0 - w.squaredNorm());
    m.covariance = 0.5 * (m.covariance + m.covariance.transpose());
    return m;
}
```

9.3 Allocation, budget schedule and volatility targeting

Budget schedule. The budgets in force at rebalance row t are the last *budget change* with first row at or before t. A schedule row dated d takes effect from the first returns row dated on or after d (a binary search). This is how time-varying budgets from a rule or from agents enter one continuous backtest. Without a schedule, the parameter budgets apply.

Volatility targeting. The risk budgeting weights x (summing to 1) are scaled by the leverage

$$L = \min\Big(L_{\max},\ \frac{\sigma^*}{\sqrt{x^\top \hat{\Sigma} x \cdot P}}\Big),$$

so the predicted annualized volatility is σ^* unless the cap binds. Any weight not invested, $1 - \sum_i w_i$, is cash earning zero.

Code 20: Schedule dates to first rows
`include/nabia/strategy/RiskBudgetingPortfolio.h:166-168`

```
for (Eigen::Index s = 0; s < budget_schedule.rows(); ++s) {
    const auto first = std::lower_bound(index.begin(), index.end(), budget_schedule.time(s));
    schedule.push_back({static_cast<Eigen::Index>(first - index.begin()), budget_schedule.values().row(
    ↪ s).transpose()});
```

Code 21: Budgets in force at a rebalance
`src/strategy/RiskBudgetingPortfolio.cpp:164-174`

```
// Budgets in force at a rebalance row: the last change at or before it.
auto budgets_at = [&](Index row) -> const VectorXd& {
    const VectorXd* b = &default_budgets;
    for (const BudgetChange& change : schedule) {
        if (change.first_row > row) {
            break;
        }
        b = &change.budgets;
    }
    return *b;
};
```

Code 22: Target weights of one rebalance date
`src/strategy/RiskBudgetingPortfolio.cpp:92-106`

```
Target solve_target(const RiskBudgetingPortfolioParams& p, const VectorXd& budgets, const MatrixXd&
    ↪ window,
                    const std::optional<VectorXd>& warm_start) {
    const WindowMoments m = estimate(p, window);
    const RiskBudgetingResult rb = allocate(p, budgets, m, warm_start);
    Target target;
    if (p.target_volatility) {
        const double ppy = static_cast<double>(p.periods_per_year);
        const double predicted = std::sqrt(std::max(0.0, rb.weights.dot(m.covariance * rb.weights)) *
    ↪ ppy);
        target.leverage = predicted > 0.0 ? std::min(p.max_leverage, *p.target_volatility / predicted)
    ↪ : p.max_leverage;
    }
    target.weights = target.leverage * rb.weights;
    target.contributions = rb.relative_contributions;
    target.converged = rb.converged;
    return target;
}
```

Code

risk measure selection — `src/strategy/RiskBudgetingPortfolio.cpp:67-79`

9.4 Phase 1: parallel solves

The targets of all rebalance dates depend only on their own windows, so they are solved first, in parallel. Each solve warm-starts from the previous date's unlevered weights, which cuts iterations sharply because consecutive windows differ by a few rows. Warm starts make solves depend on each other, so dates are grouped into *fixed* blocks of 8 consecutive rebalances:

- one block is one task;
- warm starts happen only within a block;
- the block boundaries do not depend on the thread count.

Results are therefore bit-identical for any number of threads. With 50 assets, daily rebalancing and 10 years of data (2,271 solves), a backtest takes 375 ms on one thread and 53 ms on eight.

Code 23: Phase 1

`src/strategy/RiskBudgetingPortfolio.cpp:192-208`

```
// Phase 1 (parallel): target weights of every rebalance date. Each task
// solves one block of consecutive dates, warm-starting within the block.
std::vector<Target> targets(static_cast<std::size_t>(n_rebalances));
const Index n_blocks = (n_rebalances + kWarmStartBlock - 1) / kWarmStartBlock;
const int threads = nabia::detail::resolve_threads(p.num_threads, n_blocks);
nabia::detail::parallel_for(n_blocks, threads, [&](std::int64_t block) {
    std::optional<VectorXd> warm_start;
    const Index first = block * kWarmStartBlock;
    const Index last = std::min(n_rebalances, first + kWarmStartBlock);
    for (Index k = first; k < last; ++k) {
        const Index t = c.rebalance_rows[static_cast<std::size_t>(k)];
        Target target = solve_target(p, budgets_at(t), returns.middleRows(t - W + 1, W), warm_start);
        warm_start = target.weights / target.leverage;  // the unlevered risk budgeting weights
        targets[static_cast<std::size_t>(k)] = std::move(target);
    }
});
c.threads_used = threads;
```

Code 24: Thread pool helper

`src/common/parallel_for.hpp:27-57`

```
void parallel_for(std::int64_t count, int threads, const Task& task) {
    std::atomic<std::int64_t> next{0};
    std::mutex error_mutex;
    std::exception_ptr error;
    std::int64_t error_index = count;

    auto work = [&] {
        for (std::int64_t i = next++; i < count; i = next++) {
            try {
                task(i);
            } catch (...) {
                const std::lock_guard lock(error_mutex);
                if (i < error_index) {
                    error_index = i;
                    error = std::current_exception();
                }
            }
        }
    };
    {
        std::vector<std::jthread> workers;
        workers.reserve(static_cast<std::size_t>(std::max(0, threads - 1)));
        for (int t = 1; t < threads; ++t) {
```

```
            workers.emplace_back(work);
        }
        work();
    }  // joining the workers makes all their writes visible here
    if (error) {
        std::rethrow_exception(error);
    }
}
```

Code

thread count (0 = hardware concurrency, capped by tasks) — `src/common/parallel_for.hpp:16-20`

9.5 Phase 2: drift, trading and accounting

Phase 2 walks through time. With held weights w_t and asset returns r_t, each row does the following:

$$
\begin{aligned}
g_t &= w_t^\top r_t && \text{gross return,}\\
w &\leftarrow w_t \odot (1 + r_t)/(1 + g_t) && \text{drift,}\\
\tau_t &= \|w^{\text{target}} - w\|_1, \quad c_t = \kappa\,\tau_t && \text{turnover and cost (rebalance rows),}\\
n_t &= (1 + g_t)(1 - c_t) - 1, \quad E_t = E_{t-1}(1 + n_t) && \text{net return and equity.}
\end{aligned}
$$

Drift. Holding i is worth $w_i E$ and grows to $w_i E(1 + r_i)$. Cash earns nothing, so equity grows to $E(1 + g)$. The new weight is the ratio of the two.

Costs. They are paid out of equity at the close, after the period's return, which gives $E(1 + g)(1 - c)$. The first trade buys the whole portfolio from cash, so its turnover is $\sum_i w_i$.

Checks. The engine stops with an error if $1 + g_t \le 0$ (the portfolio lost all its capital). These identities are checked to 10^{-15} in the tests.

Code 25: Phase 2
`src/strategy/RiskBudgetingPortfolio.cpp:210-254`

```
// Phase 2 (sequential): drift, trade and account through time.
VectorXd w = VectorXd::Zero(N);  // current weights (fractions of equity)
double equity = p.initial_capital;
const double ppy = static_cast<double>(p.periods_per_year);
Index next = 0;  // next rebalance

for (Index t = 0; t < T; ++t) {
    // 1. Earn this period's returns with the weights set at the previous close.
    c.weights.row(t) = w.transpose();
    const VectorXd r = returns.row(t).transpose();
    const double gross = w.dot(r);
    if (!(1.0 + gross > 0.0)) {
        throw std::invalid_argument("RiskBudgetingPortfolio: the portfolio lost all its capital at row
    ↪ " +
                                    std::to_string(t));
    }
    if (p.drift_weights) {
        // Each holding grows by (1 + r_i), equity by (1 + gross); cash is the remainder.
        w = w.cwiseProduct((VectorXd::Ones(N) + r)) / (1.0 + gross);
    }

    // 2. Rebalance at the close, with the data known at t.
    double cost = 0.0;
    if (next < n_rebalances && c.rebalance_rows[static_cast<std::size_t>(next)] == t) {
        const Target& target = targets[static_cast<std::size_t>(next)];
        const double traded = (target.weights - w).lpNorm<1>();
        cost = p.transaction_cost * traded;
        w = target.weights;
```

```
        c.target_weights.row(next) = target.weights.transpose();
        c.risk_contributions.row(next) = target.contributions.transpose();
        c.leverage(next) = target.leverage;
        c.all_converged = c.all_converged && target.converged;
        c.turnover(t) = traded;
        ++next;
    }

    // 3. Accounting: costs are paid out of the equity at the close.
    const double net = (1.0 + gross) * (1.0 - cost) - 1.0;
    const double new_equity = equity * (1.0 + net);
    c.gross_returns(t) = gross;
    c.costs(t) = cost;
    c.net_returns(t) = net;
    c.pnl(t) = new_equity - equity;
    c.equity(t) = new_equity;
    equity = new_equity;
```

10 Performance metrics

The metrics use the net returns $n_1, \ldots, n_T$ from the first trade on, with P periods per year and equity E starting at 1.

Metric	Definition
Total return	$\prod_t (1 + n_t) - 1$
Annualized return (CAGR)	$(1 + \text{total})^{P/T} - 1$
Annualized volatility	$\text{sd}(n)\sqrt{P}$ (sample standard deviation)
Sharpe ratio	$\bar{n}\,P/(\text{sd}(n)\sqrt{P})$ (zero risk-free rate)
Sortino ratio	$\bar{n}\,P/\big(\sqrt{\frac{1}{T}\sum_t \min(n_t, 0)^2}\,\sqrt{P}\big)$
Maximum drawdown	$\max_t \big(1 - E_t / \max_{s \le t} E_s\big)$
Max drawdown duration	longest run of periods below a previous peak
Calmar ratio	annualized return / maximum drawdown
Hit rate	fraction of periods with $n_t > 0$
Best / worst period	$\max_t n_t$, $\min_t n_t$
Turnover	average τ per rebalance; annualized $\sum_t \tau_t\,P/T$
Total costs	$\sum_t c_t$

Code 26: Performance metrics
`src/strategy/PerformanceMetrics.cpp:31-67`

```
PerformanceMetrics compute_performance(const Eigen::VectorXd& returns, int periods_per_year) {
    if (returns.size() == 0 || !returns.allFinite()) {
        throw std::invalid_argument("compute_performance: returns must be non-empty and finite");
    }
    if (periods_per_year <= 0) {
        throw std::invalid_argument("compute_performance: periods_per_year must be positive");
    }
    const auto n = static_cast<double>(returns.size());
    const double ppy = static_cast<double>(periods_per_year);
    PerformanceMetrics m;
    m.periods = returns.size();

    const Eigen::VectorXd equity = equity_curve(returns, 1.0);
    m.total_return = equity(equity.size() - 1) - 1.0;
    m.annualized_return = m.total_return > -1.0 ? std::pow(1.0 + m.total_return, ppy / n) - 1.0 : -1.0;

    const double mean = returns.mean();
    const double var = returns.size() > 1 ? (returns.array() - mean).square().sum() / (n - 1.0) : 0.0;
    m.annualized_volatility = std::sqrt(var * ppy);
    m.sharpe_ratio = m.annualized_volatility > 0.0 ? mean * ppy / m.annualized_volatility : 0.0;
    const double downside = std::sqrt(returns.cwiseMin(0.0).squaredNorm() / n) * std::sqrt(ppy);
    m.sortino_ratio = downside > 0.0 ? mean * ppy / downside : 0.0;
```

```
    const Eigen::VectorXd dd = drawdown_series(equity, 1.0);
    m.max_drawdown = dd.maxCoeff();
    Eigen::Index run = 0;
    for (Eigen::Index t = 0; t < dd.size(); ++t) {
        run = dd(t) > 0.0 ? run + 1 : 0;
        m.max_drawdown_duration = std::max(m.max_drawdown_duration, run);
    }
    m.calmar_ratio = m.max_drawdown > 0.0 ? m.annualized_return / m.max_drawdown : 0.0;

    m.hit_rate = static_cast<double>((returns.array() > 0.0).count()) / n;
    m.best_period = returns.maxCoeff();
    m.worst_period = returns.minCoeff();
    return m;
}
```

Code

drawdown series — `src/strategy/PerformanceMetrics.cpp:21-29`

11 Elastic net factor regression

The regression module regresses a target series (a fund, a strategy, an asset) on candidate factors. It chooses the factors, the penalty and the mix, and measures their uncertainty. Typical uses are replicating a fund with tradable factors, explaining a strategy's returns, and hedging. The standalone note `doc/regression/elastic_net_regression.pdf` has every proof. This section states the results and maps them to the code.

11.1 The elastic net

For a design $X \in \mathbb{R}^{n\times p}$ and a target y, with $\ell_1 = \lambda\alpha$ and $\ell_2 = \lambda(1-\alpha)$, the solver minimizes the glmnet objective

$$\frac{1}{2n}\|y - b_0\mathbf{1} - Xb\|^2 + \ell_1\|b\|_1 + \frac{\ell_2}{2}\|b\|_2^2.$$

$\alpha = 1$ is the lasso, $\alpha = 0$ ridge, and $\lambda = 0$ least squares.

- **Intercept.** It is unpenalized, and its optimum is $b_0 = \bar{y} - \bar{x}^\top b$. The solver works on centred data.
- **Standardization.** Each centred column is divided by its population standard deviation s_j, and the coefficients are reported as $b_j = \beta_j/s_j$. The penalty therefore charges a factor in proportion to its volatility, so units and leverage do not change the selection. Constant columns get $b_j = 0$.

Proposition 11.1 (Coordinate update)**.** *With working columns z_j, $v_j = \|z_j\|^2/n$ and residual r, the exact minimizer over β_j with the others fixed is*

$$\beta_j \leftarrow \frac{\mathcal{S}(\rho_j, \ell_1)}{v_j + \ell_2}, \qquad \rho_j = \tfrac{1}{n} z_j^\top r + v_j\beta_j, \qquad \mathcal{S}(z,t) = \operatorname{sign}(z)\max(|z| - t, 0).$$

Proof. In β_j alone the objective is $\frac{v_j+\ell_2}{2}\beta_j^2 - \rho_j\beta_j + \ell_1|\beta_j|$ plus a constant. Its subgradient condition $0 \in (v_j + \ell_2)\beta_j - \rho_j + \ell_1\partial|\beta_j|$ has the stated solution in each of the cases $\rho_j > \ell_1$, $\rho_j < -\ell_1$ and $|\rho_j| \le \ell_1$. □

Each update costs $O(n)$: the residual is corrected by $-z_j\Delta_j$. Cyclic coordinate descent converges because the non-smooth term is separable (Tseng, 2001).

- **Active set.** After a full sweep, the solver sweeps only the non-zero coefficients until they settle, then checks with a full sweep.

- **Stopping rule.** The solver stops when a full sweep moves no fitted value by more than a fraction τ of the target's variance: $\max_j v_j \Delta_j^2 < \tau \|\tilde{y}\|^2/n$.

Proposition 11.2 ($\lambda_{\max}$)**.** *For $\alpha > 0$, $\beta = 0$ is optimal if and only if $\lambda \geq \lambda_{\max} = \max_j |z_j^\top \tilde{y}|/(n\alpha)$.*

Proof. At $\beta = 0$ the residual is $\tilde{y}$, and the optimality condition of an inactive coefficient is $|z_j^\top r|/n \leq \ell_1$. □

- **Floor on α.** As in glmnet, α is floored at 10^{-3}, so ridge also gets a finite path.
- **Rounding.** The value is multiplied by $1 + 10^{-12}$, so rounding cannot leave a 10^{-17} coefficient at $\lambda_{\max}$.
- **Grid.** The automatic grid is $\lambda_i = \lambda_{\max} r^{i/(L-1)}$. It is solved from the largest λ down, each fit warm-started from the previous one.

Degrees of freedom. With active set A and locally constant signs, the active coefficients are an affine function of y: $\beta_A = (Z_A^\top Z_A + n\ell_2 I)^{-1}(Z_A^\top \tilde{y} - n\ell_1 s_A)$. By Stein's lemma, the degrees of freedom are the trace of the hat matrix:

$$\mathrm{df} = 1 + \mathrm{tr}\big(Z_A (Z_A^\top Z_A + n\ell_2 I)^{-1} Z_A^\top\big) = 1 + \sum_i \frac{d_i}{d_i + n\ell_2},$$

where the d_i are the eigenvalues of $Z_A^\top Z_A$ and the 1 counts the intercept. The lasso gives $|A| + 1$ (Zou, Hastie and Tibshirani, 2007).

Grouping. For standardized factors i, j with correlation ρ and coefficients of the same sign, the optimality conditions give $|\beta_i - \beta_j| \leq \|\tilde{y}\| \sqrt{2(1-\rho)}/(\sqrt{n}\,\ell_2)$. Near-duplicate factors share a loading instead of one of them taking it arbitrarily. This is why $\alpha < 1$ helps with substitutes such as the S&P 500 and the Nasdaq 100.

Code 27: One coordinate descent sweep
`src/regression/ElasticNet.cpp:124-143`

```
double sweep(bool active_only) {
    const auto n = static_cast<double>(s_.X.rows());
    double max_change = 0.0;
    for (Index j = 0; j < beta_.size(); ++j) {
        const double vj = s_.v(j);
        if (vj == 0.0 || (active_only && beta_(j) == 0.0)) {
            continue;
        }
        const double old = beta_(j);
        const double z = s_.X.col(j).dot(residual_) / n + vj * old;
        const double updated = soft_threshold(z, l1_) / (vj + l2_);
        const double delta = updated - old;
        if (delta != 0.0) {
            residual_ -= delta * s_.X.col(j);
            beta_(j) = updated;
            max_change = std::max(max_change, vj * delta * delta);
        }
    }
    return max_change;
}
```

Code 28: $\lambda_{\max}$ with the same arithmetic as the first update
`src/regression/ElasticNet.cpp:211-227`

```
double lambda_max(const MatrixXd& X, const VectorXd& y, double alpha, bool fit_intercept, bool
    ↪ standardize) {
    validate_data(X, y);
```

```
    if (!(alpha >= 0.0 && alpha <= 1.0)) {
        throw std::invalid_argument("lambda_max: alpha must be in [0, 1]");
    }
    if (X.cols() == 0) {
        return 0.0;
    }
    const Standardized s = standardize_data(X, y, fit_intercept, standardize);
    // Same arithmetic as the first coordinate update, plus a few ulps so that
    // lambda * alpha >= |gradient| survives the rounding of the division.
    double gradient = 0.0;
    for (Index j = 0; j < X.cols(); ++j) {
        gradient = std::max(gradient, std::abs(s.X.col(j).dot(s.y) / static_cast<double>(X.rows())));
    }
    return gradient / std::max(alpha, kMinAlphaForLambdaMax) * (1.0 + 1e-12);
}
```

Code

parameters — `include/nabia/regression/ElasticNet.h:11-18`
centring and scaling — `src/regression/ElasticNet.cpp:48-77`
solver loop: full and active-set sweeps — `src/regression/ElasticNet.cpp:90-150`
path with warm starts — `src/regression/ElasticNet.cpp:251-275`
effective degrees of freedom — `src/regression/ElasticNet.cpp:153-175`

11.2 The regression engine

`RegressionEngine` runs the following steps (`src/regression/RegressionEngine.cpp:472-555`):

1. **Split** the T rows in time order. The first $n = \lfloor fT + 10^{-9} \rfloor$ rows train, and the rest test. The test set is an extrapolation. With $f = 1$ the engine is in sample.
2. For each candidate subset S of $k_{\min}..k_{\max}$ factors:
   (a) choose (α, λ) by time-series cross-validation on the training rows;
   (b) fit on all training rows, warm-started down the grid;
   (c) score by the test MSE, or in sample by BIC $= n \log \mathrm{MSE} + \mathrm{df} \log n$ or AIC $= n \log \mathrm{MSE} + 2\,\mathrm{df}$, which are $-2 \log \hat{L}$ under Gaussian errors plus the penalty, with constants dropped.
3. **Keep** the lowest score. Optionally refit on all rows, and bootstrap.

Cross-validation. The training rows are cut into contiguous blocks with boundaries $\lfloor kn/B \rfloor$, and g rows are left out on each side of a validation block.

- *Blocked K-fold* ($B = K$) trains on the rows before and after the validation block.
- *Walk-forward* ($B = K + 1$) trains only on the rows before it.

The estimate is the mean fold MSE, and its standard error is $\sqrt{\widehat{\mathrm{Var}}_k(m_k)/K}$. The cell with the lowest mean wins. The one-standard-error rule then moves, within that α, to the largest λ whose error is within one standard error of the minimum, which gives a sparser model that is statistically as good.

Subset search.

- *Exhaustive* evaluates $\sum_k \binom{N}{k}$ subsets (1 470 for 14 factors and sizes 1–4).
- *Forward* adds, greedily, the factor that most improves the score, which takes $\sum_{k \le k_{\max}} (N - k + 1)$ evaluations.

- *Auto* chooses Exhaustive below `max_exhaustive_subsets`.

The test MSE of the winner of many subsets is optimistic, because a factor can win by noise. The bootstrap below exposes such factors, and a later holdout gives a clean figure.

Moving block bootstrap. Each of B samples:

1. concatenates training-row blocks of length ℓ with uniform starts until there are n rows;
2. refits the selected subset at the selected (α, λ);
3. records the coefficients, the intercept and the test MSE.

Blocks keep the serial dependence of returns, such as volatility clustering (Künsch, 1989). The summaries are:

- the mean and standard error;
- percentile intervals (linear interpolation at $q(B-1)$);
- selection frequencies, the share of samples with a non-zero coefficient;
- the spread of the test MSE.

The intervals are conditional on the selected model, so they do not include selection uncertainty.

Determinism. Subsets and bootstrap samples run on the thread pool of §9. Each task writes its own slot, ties go to the first index, and sample b seeds its own `mt19937_64` from `seed_seq(seed, b)`. Results are bit-identical for any thread count.

Code 29: Contiguous validation folds with a gap
`src/regression/RegressionEngine.cpp:71-94`

```
std::vector<Fold> make_folds(const RegressionEngineParams& p, Index n) {
    const Index K = p.validation_folds;
    const Index blocks = p.validation == ValidationScheme::BlockedKFold ? K : K + 1;
    const auto bound = [&](Index i) { return i * n / blocks; };
    std::vector<Fold> folds;
    const Index first = p.validation == ValidationScheme::BlockedKFold ? 0 : 1;
    for (Index k = first; k < blocks; ++k) {
        const Index begin = bound(k);
        const Index end = bound(k + 1);
        Fold f;
        f.validation = range(begin, end);
        f.train = range(0, begin - p.validation_gap);
        if (p.validation == ValidationScheme::BlockedKFold) {
            const std::vector<Index> after = range(end + p.validation_gap, n);
            f.train.insert(f.train.end(), after.begin(), after.end());
        }
        if (f.validation.empty() || f.train.size() < 2) {
            throw std::invalid_argument("RegressionEngine: " + std::to_string(n) +
                                        " training rows are too few for the validation folds and gap");
        }
        folds.push_back(std::move(f));
    }
    return folds;
}
```

Code 30: Cross-validation error, standard error and the 1-SE rule
`src/regression/RegressionEngine.cpp:144-191`

```
if (pb.tune) {
    MatrixXd sum = MatrixXd::Zero(A, L);
    MatrixXd sum_sq = MatrixXd::Zero(A, L);
    for (const Fold& fold : pb.folds) {
```

```
        const MatrixXd Xf = X(fold.train, all);
        const VectorXd yf = pb.y_train(fold.train);
        const MatrixXd Xv = X(fold.validation, all);
        const VectorXd yv = pb.y_train(fold.validation);
        for (Index a = 0; a < A; ++a) {
            const auto ai = static_cast<std::size_t>(a);
            const std::vector<ElasticNetFit> path =
                elastic_net_path(Xf, yf, solver_settings(p, p.alphas[ai]), grids[ai]);
            for (Index l = 0; l < L; ++l) {
                const ElasticNetFit& fit = path[static_cast<std::size_t>(l)];
                const double m = mse(yv, fit.predict(Xv));
                sum(a, l) += m;
                sum_sq(a, l) += m * m;
                e.converged = e.converged && fit.converged;
            }
        }
    }
    const auto K = static_cast<double>(pb.folds.size());
    const MatrixXd mean = sum / K;
    MatrixXd se = MatrixXd::Zero(A, L);
    if (pb.folds.size() > 1) {
        const MatrixXd var = ((sum_sq / K - mean.cwiseProduct(mean)) * (K / (K - 1.0))).cwiseMax(0.0);
        se = (var / K).cwiseSqrt();
    }
    for (Index a = 0; a < A; ++a) {
        for (Index l = 0; l < L; ++l) {
            if (mean(a, l) < mean(best_a, best_l)) {
                best_a = a;
                best_l = l;
            }
        }
    }
    if (p.one_standard_error_rule) {
        // Lambdas decrease along the row: the first one within the band is the largest.
        const double threshold = mean(best_a, best_l) + se(best_a, best_l);
        for (Index l = 0; l <= best_l; ++l) {
            if (mean(best_a, l) <= threshold) {
                best_l = l;
                break;
            }
        }
    }
    e.validation_mse = mean(best_a, best_l);
    e.validation_se = se(best_a, best_l);
```

Code 31: One moving block bootstrap sample
`src/regression/RegressionEngine.cpp:364-378`

```
// One generator per sample, seeded by (seed, b): independent of the thread count.
std::seed_seq seq{static_cast<std::uint32_t>(p.bootstrap_seed), static_cast<std::uint32_t>(p.
    ↪ bootstrap_seed >> 32),
                  static_cast<std::uint32_t>(b), static_cast<std::uint32_t>(static_cast<std::uint64_t>(
    ↪ b) >> 32)};
std::mt19937_64 rng(seq);
std::uniform_int_distribution<Index> start(0, n - block);
std::vector<Index> rows;
rows.reserve(static_cast<std::size_t>(n));
while (static_cast<Index>(rows.size()) < n) {
    const Index s0 = start(rng);
    for (Index i = 0; i < block && static_cast<Index>(rows.size()) < n; ++i) {
        rows.push_back(s0 + i);
    }
}
const ElasticNetFit fit =
    elastic_net_path(X(rows, all), pb.y_train(rows), settings, {best.lambda}).front();
```

Code

parameters (defaults and meaning) — `include/nabia/regression/RegressionEngine.h:39-80`
engine class, index-free and `DataFrame` interfaces — `include/nabia/regression/RegressionEngine.h:183-222`
evaluation of one subset — `src/regression/RegressionEngine.cpp:127-231`
BIC, AIC, MSE — `src/regression/RegressionEngine.cpp:106-118`

exhaustive search — `src/regression/RegressionEngine.cpp:284-308`
forward search — `src/regression/RegressionEngine.cpp:310-331`
bootstrap and its summaries — `src/regression/RegressionEngine.cpp:345-417`

Part III
Application and Python bindings

12 Command-line application

`nabia_rb_backtest --prices prices.csv --config config.json --output out.csv`:

- **Prices** come from a CSV with ISO dates and one column per asset. Empty cells are missing values.
- **Alignment**: dates where any asset is missing are dropped, and the drop is reported.
- **Returns**: simple returns are computed after alignment.
- **Parameters** come from a JSON file with the keys of Table 1, plus an optional `start` and `end`. Unknown keys are rejected, so a typo fails loudly. Budgets can be a list or an object keyed by asset name.
- **Output**: a CSV row per date with the weights, the risk contributions on rebalance dates, turnover, costs, returns, PnL, equity and drawdown, and an optional metrics CSV.

Code

entry point — `apps/rb_backtest/main.cpp:94-139`
JSON parser — `apps/rb_backtest/config.cpp:68-163`
CSV reader — `apps/rb_backtest/csv_io.cpp:156-214`
CSV writer — `apps/rb_backtest/csv_io.cpp:216-259`

13 Backtest bindings for the Python package

The module `nabia_python_bridge._rb_backtest` reuses the application's pipeline and JSON parser, so the command line and Python accept the same parameters with the same error messages. The `nabia` package of the Python project wraps it in pandas objects (`nabia.backtest`, `nabia.latest_weights`; *Portfolio backtests* (Python reference)). Data crosses the boundary in simple forms:

- dates as `int64` days since 1970-01-01 (numpy `datetime64[D]`);
- prices as a `float64` matrix;
- parameters as JSON text.

The GIL is released during the computation, so the engine's threads run in parallel. `latest_weights` solves only the last rebalance; the agents use it to turn budgets into target weights.

Code 32: Shared preparation: parse, align, window, returns, budgets
`bindings/src/rb_backtest_module.cpp:65-96`

```
Prepared prepare(const DaysArray& days, const std::vector<std::string>& assets, const MatrixXd& prices,
                 const std::string& config_json) {
    if (prices.rows() != days.size() || prices.cols() != static_cast<Index>(assets.size())) {
```

```
        throw std::invalid_argument("prices must be (dates x assets): got " + std::to_string(prices.
    ↪ rows()) + " x " +
                                    std::to_string(prices.cols()) + " for " + std::to_string(days.size
    ↪ ()) +
                                    " dates and " + std::to_string(assets.size()) + " assets");
    }
    Prepared p;
    p.config = nabia::app::parse_config(config_json);
    DataFrame<Date> px(to_dates(days), assets, prices);
    const Index read = px.rows();
    px = px.drop_missing();
    p.dropped_dates = read - px.rows();
    if (px.rows() >= 2 && (p.config.start || p.config.end)) {
        px = px.between(p.config.start.value_or(px.time(0)), p.config.end.value_or(px.time(px.rows() -
    ↪ 1)));
    }
    if (px.rows() < 2) {
        throw std::invalid_argument("need at least 2 dates with a price for every asset");
    }
    for (Index t = 0; t < px.rows(); ++t) {
        for (Index j = 0; j < px.cols(); ++j) {
            if (!(px.values()(t, j) > 0.0)) {
                throw std::invalid_argument("prices must be positive for returns: " + px.columns()[
    ↪ static_cast<std::size_t>(j)] +
                                            " is " + std::to_string(px.values()(t, j)) + " on " +
                                            nabia::app::format_date(px.time(t)));
            }
        }
    }
    p.returns = px.pct_change();
    p.config.params.budgets = nabia::app::resolve_budgets(p.config, assets);
    return p;
}
```

Code

`backtest` binding — `bindings/src/rb_backtest_module.cpp:119-160`
`latest_weights` binding — `bindings/src/rb_backtest_module.cpp:162-189`
module definition — `bindings/src/rb_backtest_module.cpp:193-206`

14 The full library in Python: `nabia_python_bridge`

The pybind11 package `nabia_python_bridge` exposes *every* public class and function of the C++ library, for research and scripting from Python. Its sources are in `bindings`: the C++ of the modules in `src`, and the Python files of the package (re-exports and the pandas interface `frames`) in `nabia_python_bridge`.

14.1 Modules

Module	Contents
`stats`	`mean`, `variance`, `stddev`, `autocorrelation`; `normal_pdf/cdf/quantile` (vectorized)
`signal`	`EWMA`, `ButterworthBandPass` (streaming `process`, batch `apply`), `moving_average`, `exponential_moving_average`, `diff`
`forecast`	`KalmanFilter` (local level, local linear trend, custom), `GaussianProcess` with squared exponential, Matérn 5/2, periodic, sum and product kernels (`k1 + k2`, `k1 * k2`), `SimpleExponentialSmoothing`, `Forecast`
`optimization`	`LBFGS`, `AugmentedLagrangian`, `SQP`, `NelderMead`, `DifferentialEvolution` with Python objectives; `BoxConstraints`, `LinearConstraint`
`risk`	`Volatility`, `ValueAtRisk`, `ExpectedShortfall`, `RiskBudgeting`
`strategy`	`RiskBudgetingPortfolio`, `compute_performance`, `equity_curve`, `drawdown_series`
`regression`	`ElasticNet`, `elastic_net_path`, `lambda_max`, `lambda_grid`, `RegressionEngine` (§11)
`frames`	pandas interface: `backtest`, `regress`, `performance`, `metrics_dict`

The `TimeSeries` and `DataFrame` containers are not bound, because pandas does their job in Python. Where an engine takes indexed data, `frames` maps the pandas index to row positions and back.

14.2 Design

Parameters as keyword arguments. Every parameter struct is built from keyword arguments, and a model accepts the fields directly: `signal.EWMA(alpha=0.2)`. Each keyword is assigned through the bound attribute, so the attribute's type checks apply, and an unknown name raises `AttributeError` instead of being silently ignored. `lambda` is a Python keyword, so it is spelled `lambda_`.

Arrays. Vectors and matrices are `float64` numpy arrays. Matrices have one row per observation, as in the C++ code. Results are copies, so they stay valid after the next run.

Errors. `std::invalid_argument` becomes `ValueError`, and "not fitted" (`std::logic_error`) becomes `RuntimeError`.

GIL. Fits, backtests, regressions and optimizations release the GIL, so the engines' own threads run in parallel. A Python objective is wrapped in a `PyCallable`. Copies of it share one pointer, and every call, and the final release, takes the GIL. C++ can therefore call it from any thread. It is only faster with threads if the objective itself releases the GIL (numpy on large arrays). A Python exception raised in an objective propagates out of `optimize()`.

Templates. `RiskBudgeting<Measure>` is a template. The binding wraps its three instantiations in one `AnyRiskBudgeting` type (a `std::variant`), so Python sees a single `RiskBudgeting` class that accepts any of the three measures.

Kernels are polymorphic and owned by `unique_ptr` in C++. In Python they are shared objects. `GaussianProcessParams` clones the kernel when it is set, so fitting never modifies the user's kernel.

Readable results. The Python package re-exports each submodule and gives every result struct a `__repr__` listing its fields.

Code 33: Parameter structs from keyword arguments
`bindings/src/bridge.hpp:27-36`

```
std::unique_ptr<T> from_kwargs(const py::kwargs& kwargs) {
    auto value = std::make_unique<T>();
    {
        const py::object self = py::cast(value.get(), py::return_value_policy::reference);
        for (const auto& [name, item] : kwargs) {
            py::setattr(self, name, item);
        }
    }
    return value;
}
```

Code 34: A Python callable usable from any C++ thread
`bindings/src/bridge.hpp:61-77`

```
class PyCallable {
public:
    explicit PyCallable(py::function f)
        : f_(new py::function(std::move(f)), [](py::function* p) {
              const py::gil_scoped_acquire gil;
              delete p;
          }) {}
```

```
    template <typename R, typename... Args>
    R call(const Args&... args) const {
        const py::gil_scoped_acquire gil;
        return (*f_)(args...).template cast<R>();
    }

private:
    std::shared_ptr<py::function> f_;
};
```

14.3 pandas interface

- `frames.backtest(returns_df, params=None, budget_schedule=None, **fields)` runs `RiskBudgetingPortfolio`. Budgets may be a mapping by column name, and the schedule is a DataFrame indexed by date. It returns weights, target weights, risk contributions, leverage, a performance frame and the metrics, all labelled.
- `frames.regress(target, factors_df, params=None, **fields)` aligns the target and the factors on their common index and drops rows with a missing value. It then runs the regression engine and returns the coefficients by factor name, predictions and residuals by date, the first out-of-sample date, the best subset of each size and the bootstrap table.
- In both, enum fields also accept member names (`risk_measure="ValueAtRisk"`, `validation="WalkForward"`), and the given parameters object is copied, never modified.

Code 35: `frames.regress`: alignment, engine and labelled results
`bindings/nabia_python_bridge/frames.py:125-177`

```
def regress(target: pd.Series, factors: pd.DataFrame, params: Any = None, **overrides: Any) ->
    ↪ RegressionFrames:
    """Elastic net regression of ‘target‘ on ‘factors‘ with factor, lambda and alpha selection.

    The two are aligned on their common index and rows with a missing value
    are dropped. Parameters are a ‘‘regression.RegressionEngineParams‘‘ and/or
    its fields as keyword arguments, e.g. ‘‘min_factors=1, max_factors=3,
    train_fraction=0.7, bootstrap_samples=500‘‘.
    """
    if not isinstance(factors, pd.DataFrame):
        raise TypeError("factors must be a pandas DataFrame")
    data = factors.join(pd.Series(target, name="__target__"), how="inner").dropna()
    names = [str(c) for c in factors.columns]
    p = _make_params(_regression.RegressionEngineParams, params, overrides)
    y = data.pop("__target__").to_numpy(dtype=float)
    c = _regression.RegressionEngine(p).run(y, data.to_numpy(dtype=float), names)

    index = data.index
    sizes = pd.DataFrame([
        {
            "size": len(e.factors),
            "factors": [names[j] for j in e.factors],
            "alpha": e.alpha,
            "lambda": e.lambda_,
            "nonzero": e.nonzero,
            "validation_mse": e.validation_mse,
            "train_mse": e.train_mse,
            "train_r2": e.train_r2,
            "test_mse": e.test_mse,
            "test_r2": e.test_r2,
            "criterion": e.criterion,
            "score": e.score,
        }
        for e in c.best_by_size
    ]).set_index("size")
    boot = None
    if c.bootstrap is not None:
        b = c.bootstrap
```

```
        boot = pd.DataFrame(
            {"mean": b.mean, "std_error": b.std_error, "lower": b.lower, "upper": b.upper,
             "selection_frequency": b.selection_frequency},
            index=pd.Index(c.factor_names, name="factor"))
    return RegressionFrames(
        coefficients=pd.Series(c.coefficients, index=pd.Index(c.factor_names, name="factor"), name="
    ↪ coefficient"),
        intercept=c.intercept,
        alpha=c.alpha,
        lambda_=c.lambda_,
        predictions=pd.Series(c.predictions, index=index, name="predictions"),
        residuals=pd.Series(c.residuals, index=index, name="residuals"),
        test_start=index[c.train_rows] if c.test_rows > 0 else None,
        best_by_size=sizes,
        bootstrap=boot,
        computation=c,
    )
```

Code

module definition — `bindings/src/module.cpp:6-26`
`RiskBudgeting` over any measure — `bindings/src/bind_risk.cpp:21-57`
Python objectives for the optimizers — `bindings/src/bind_optimization.cpp:24-26`
budget schedule from rows and budgets — `bindings/src/bind_strategy.cpp:20-39`
regression bindings — `bindings/src/bind_regression.cpp:14-177`
re-export and `__repr__` — `bindings/nabia_python_bridge/_reexport.py:46-54`
parameters plus overrides — `bindings/nabia_python_bridge/frames.py:19-29`
`frames.backtest` — `bindings/nabia_python_bridge/frames.py:56-100`

14.4 Development build and tests

- **Development build.** The preset `python-bridge` builds `_core` and `_rb_backtest` in RelWithDebInfo, with the interpreter of `../python/.venv`. The modules are written next to the package's Python files, in the folder `bindings/nabia_python_bridge`, so a script run with `PYTHONPATH=cpp/bindings` imports them. Attaching CLion's debugger to that Python process stops in the C++ code.
- **Wheel.** `pyproject.toml` builds the package with scikit-build-core for each Python version and platform (§14.5).
- **Tests.** The bridge tests are Python tests, so they live in the Python project, in `python/tests/bridge`. The wheel scripts run them on every wheel they build. They check each module against closed forms and numpy, and the binding semantics:
  - Python objectives called from differential evolution's threads;
  - exceptions propagating out of objectives;
  - unknown parameter names rejected;
  - kernel composition;
  - frame alignment and budget schedules;
  - that the examples of the Python project use every exported name.

14.5 Wheels for each Python version and platform

A wheel is the installable binary package: the Python sources of `nabia_python_bridge` plus the compiled `_core` and `_rb_backtest` modules. Because they are compiled against one CPython ABI and one platform, a wheel only installs where its tag matches. For example, `cp312-cp312-macosx_13_0_arm64` needs CPython 3.12 on Apple silicon running macOS 13 or later. Supporting several versions therefore means building one wheel per Python version and platform.

How a wheel is assembled. The build backend is scikit-build-core, configured in `cpp/pyproject.toml`:

- **CMake runs on this project** (`cmake.source-dir = "."`) in `Release`, with only the bindings enabled (`NABIA_BUILD_PYTHON_BRIDGE`).
- **CMake installs the whole package:** the two modules and the `*.py` sources of the package folder in `bindings`, while `wheel.packages` is empty. A development build's modules or a `__pycache__` therefore never reach the wheel.
- **The C++ dependencies are linked statically.** nabia, iir1 and the application's parser are compiled as position-independent static libraries, and Eigen is header-only. The module therefore needs only the system C++ runtime; numpy (and pandas for `frames`) are installed by pip.
- **macOS deployment target.** `MACOSX_DEPLOYMENT_TARGET` (13.0 in the script and in CI) sets the oldest macOS the wheel supports. Without it, the tag is the macOS of the build machine. Apple's libc++ has the floating-point `std::from_chars`/`std::to_chars` only from macOS 26 and 13.3, so the application's CSV code, which is part of the wheel, formats and parses doubles with `snprintf`/`strtod` (same output, checked on 3×10^5 values).

One wheel for the current interpreter. From the repository root, with the second form setting the oldest supported macOS:

```
python/.venv/bin/python -m pip wheel ./cpp --no-deps -w dist
MACOSX_DEPLOYMENT_TARGET=13.0 python/.venv/bin/python -m pip wheel ./cpp --no-deps -w dist
```

One wheel per Python version on this machine. The script `build_wheels.sh` in `cpp/scripts` uses uv:

- it finds each interpreter, installed or downloaded by uv, preferring system installations;
- it rejects pre-release interpreters;
- it builds the wheel in an isolated environment;
- it installs the wheel in a fresh virtual environment and runs the bridge tests of the Python project (`python/tests/bridge`) from outside the source tree, so they import the installed package;
- it copies the wheel to `dist/`.

```
cpp/scripts/build_wheels.sh                     # CPython 3.10 to 3.14
cpp/scripts/build_wheels.sh 3.12 3.13           # only these versions
NABIA_WHEEL_TEST=0 cpp/scripts/build_wheels.sh  # build without testing
NABIA_WHEEL_BUILD_TYPE=RelWithDebInfo cpp/scripts/build_wheels.sh 3.14   # with debug symbols
NABIA_WHEEL_DIR=/tmp/wheels MACOSX_DEPLOYMENT_TARGET=14.0 \
    cpp/scripts/build_wheels.sh                 # other output folder and macOS target
```

On Apple silicon this produces one wheel for each of 3.10, 3.11, 3.12, 3.13 and 3.14, named `nabia_python_bridge-0.1.0-cp3XY-cp3XY-macosx_13_0_arm64.whl`. Each one passes the 34 tests.

Every platform: cibuildwheel. `[tool.cibuildwheel]` in `pyproject.toml` configures the builds:

- CPython 3.10–3.14;
- Linux `manylinux_2_28` (GCC with C++20) for x86_64 and aarch64, macOS arm64 and x86_64, Windows AMD64;
- musllinux and 32-bit builds are skipped;
- each wheel is tested with the bridge tests of `python/tests/bridge` after it is built.

Locally (Linux wheels need Docker):

```
pipx run cibuildwheel==4.3.4 --platform linux cpp     # from the repository root
pipx run cibuildwheel==4.3.4 --platform macos cpp
CIBW_BUILD="cp312-*" CIBW_ARCHS=aarch64 \
    pipx run cibuildwheel==4.3.4 --platform linux cpp   # a single wheel
pipx run cibuildwheel==4.3.4 --platform linux --print-build-identifiers cpp
```

In CI, `.github/workflows/wheels.yml` runs cibuildwheel on GitHub runners: Ubuntu x86_64 and arm64, macOS 14 for arm64 and cross-compiled x86_64, and Windows. It runs on demand, on `v*` tags, and on pull requests that change `cpp/` or the bridge tests. The wheels are uploaded as build artifacts:

```
gh workflow run wheels.yml                # start a build
gh run download --pattern 'wheels-*' -D wheelhouse   # fetch the wheels of the latest run
```

Installing and using a wheel. The Python projects install it with the script `setup_env.sh` in their `scripts` folder. By hand, pip picks the file matching the interpreter and platform from a folder of wheels:

```
pip install dist/nabia_python_bridge-0.1.0-cp312-cp312-macosx_13_0_arm64.whl
pip install --no-index --find-links dist nabia_python_bridge   # chooses the right one
pip install "nabia_python_bridge[pandas]" --find-links dist     # with pandas for frames
python -c "import nabia_python_bridge as nb; print(nb.stats.mean([1.0, 2.0, 3.0]))"
```

Limits.

- **sdist.** The wheel's CMake project is `cpp/` itself, so an sdist of `cpp/` builds anywhere with a C++20 compiler (build folders are excluded).
- **No stable ABI.** pybind11 modules use the full CPython API, so there is no `abi3` wheel and each new Python version needs its own build.
- **Verified so far:** the macOS arm64 wheels, built and tested locally. The Linux, Intel macOS and Windows wheels are configured in cibuildwheel and CI but have not been built yet. Windows (MSVC) is the most likely to need fixes.

Part IV
Verification and limitations

15 Verification

Catch2 104 test cases, plus an end-to-end run of the application on its example.

- Results against closed forms: inverse volatility for uncorrelated equal budgets, Gaussian VaR and ES, SES and Kalman steady states, filter gains at the band edges.
- Optimizer results against known optima, with constraints.
- Accounting identities to 10^{-15}.
- The elastic net against OLS, the ridge closed form and its optimality conditions. The regression engine recovering known factors (exhaustive, forward, in-sample BIC), and bootstrap intervals covering the true coefficients.
- No look-ahead, thread-count independence, and invalid input.
- The application's JSON and CSV parsing and writing.

The suites run in debug, under AddressSanitizer with UndefinedBehaviorSanitizer, and the multithreaded code under ThreadSanitizer.

Bindings the 34 bridge tests of the Python project run on every wheel built by `scripts/build_wheels.sh` and by cibuildwheel.

This document Its excerpts and code locations are extracted at build time. A listed function that no longer exists stops the build.

16 Limitations and future work

- **Gaussian risk.** VaR and ES are Gaussian; fat tails are not modelled. A historical or Cornish–Fisher measure would fit the same interface.
- **Factor regression** has constant coefficients, and its bootstrap is conditional on the selected model (§11). Exhaustive search grows as $\binom{N}{k}$.
- **SQP** solves its QP subproblems with the augmented Lagrangian to `qp_tolerance`. On some strongly convex mean-variance problems it was seen to stall at its iteration limit, so prefer the augmented Lagrangian there until the QP step is made exact (an active-set QP).
- **Wheels** are verified on macOS arm64 only; Linux, Intel macOS and Windows are configured in cibuildwheel and CI but not yet built.